

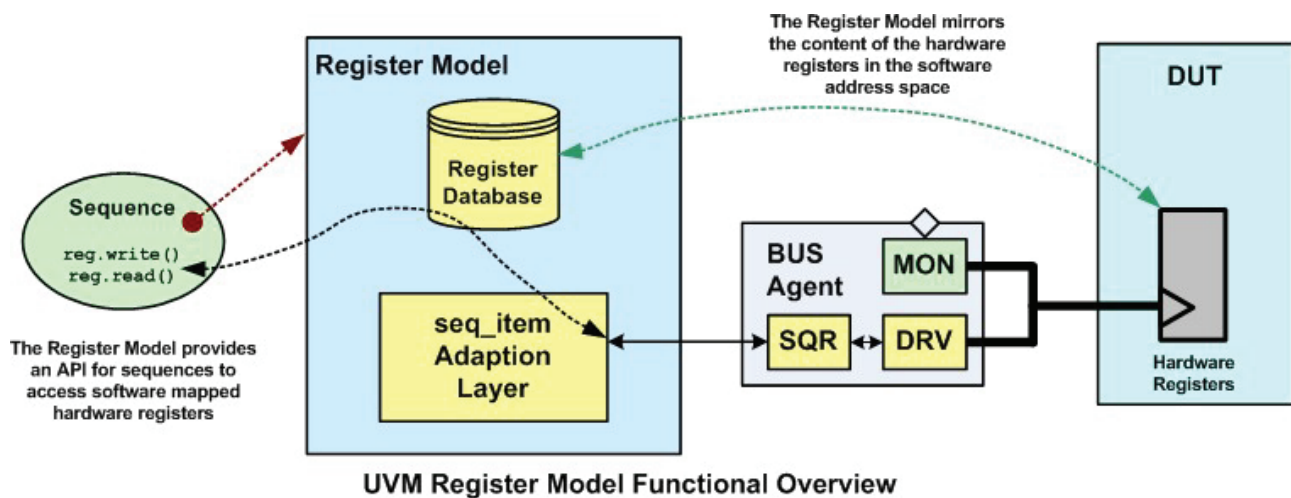
Register Abstraction Layer

The UVM Register Package

Topic Overview

Introduction

The UVM register model provides a way of tracking the register content of a DUT and a convenience layer for accessing register and memory locations within the DUT.



The register model abstraction reflects the structure of a hardware-software register specification, since that is the common reference specification for hardware design and verification engineers, and it is also used by software engineers developing firmware layer software. It is very important that all three groups reference a common specification and it is crucial that the design is verified against an accurate model.

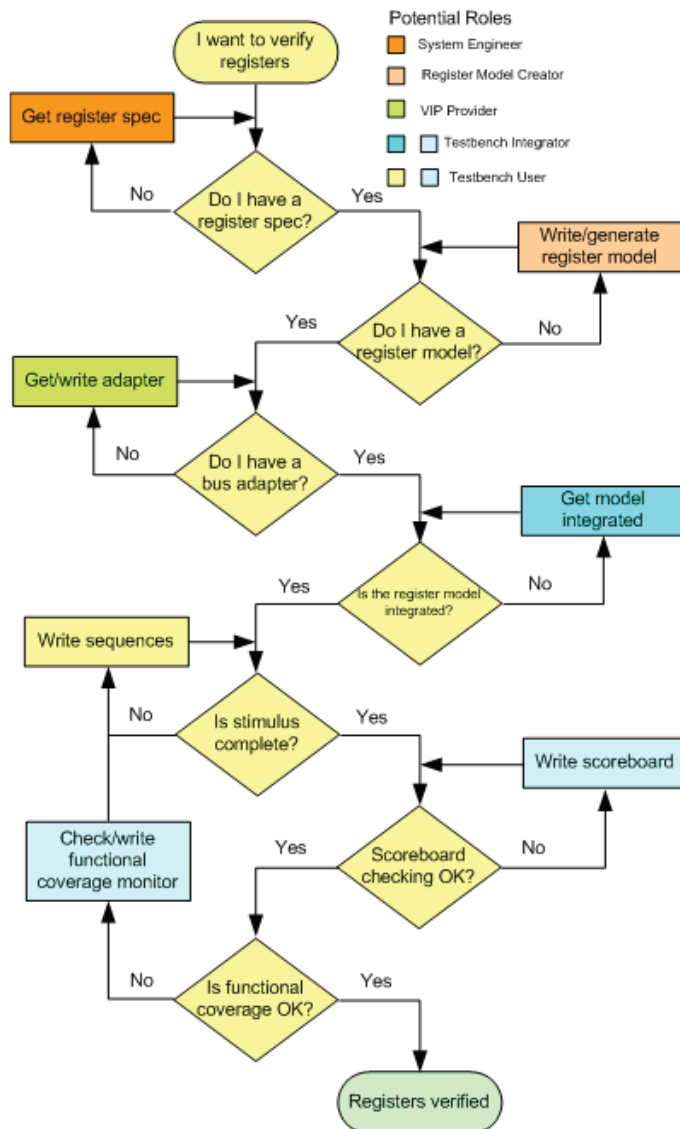
The UVM register model is designed to facilitate productive verification of programmable hardware. When used effectively, it raises the level of stimulus abstraction and makes the resultant stimulus code straight-forward to reuse, either when there is a change in the DUT register address map, or when the DUT block is reused as a sub-component.

How The UVM Register Material Is Organized

The UVM register model can be considered from several different viewpoints and this page is separated into different sections so that you can quickly navigate to the material that concerns you most. The following diagram summarizes the various steps in the flow for using the register model and outlines the different categories of users.

Therefore, the different register viewpoints are:

- The VIP developer
- The Register Model writer
- The Testbench Integrator
- The Testbench User



Register Model – Use Model Flow and Potential Roles

VIP Developer Viewpoint

In order to support the use of the UVM register package, the developer of an On Chip Bus verification component needs to develop an adapter class. This adapter class is responsible for translating between the UVM register packages generic register sequence_items and the VIP specific sequence_items. Developing the adapter requires knowledge of the target bus protocol and how the different fields in the VIP sequence_item relate to that protocol.

Once the adapter is in place it can be used by the testbench developer to integrate the register model into the UVM testbench.

To understand how to create an adapter the suggested route through the register material is:

Step	Page	Description	Relevance
1	Integrating	Describes how the adaptor fits into the overall testbench architecture	Background
2	Integration	Describes in detail how the adaptor is used	Background
3	Adapter	How to implement a register adaptor, with an example	Essential
4	AdapterContext	How to provide context to an adapter, with an example	Generally Unnecessary

Creating A Register Model

A register model can be created using a register generator application or it can be written by hand. In both cases, the starting point is the hardware-software register specification and this is transformed into the model.

If you are using a generator or writing a register model based on a register specification then these topics should be followed in this order:

Step	Page	Description	Relevance	
			Using Generator	Writing Model
1	Specification	Overview of Register Specification	Background	Background
2	RegisterModelOverview	Register Model Hierarchy Overview	Useful background	Essential
3	ModelStructure	Implementing a register model	Background	Essential
4	ComplexAddressMaps	Advanced register model addressing considerations	Background	Essential
5	QuirkyRegisters	Implementing 'Quirky' registers	Essential	Essential
6	ModelCoverage	Adding coverage models	Background	Essential
7	BackdoorAccess	Using and specifying back door accesses	Background	Essential
8	Generation	Generating a register model	Essential	Unnecessary

Integrating A Register Model

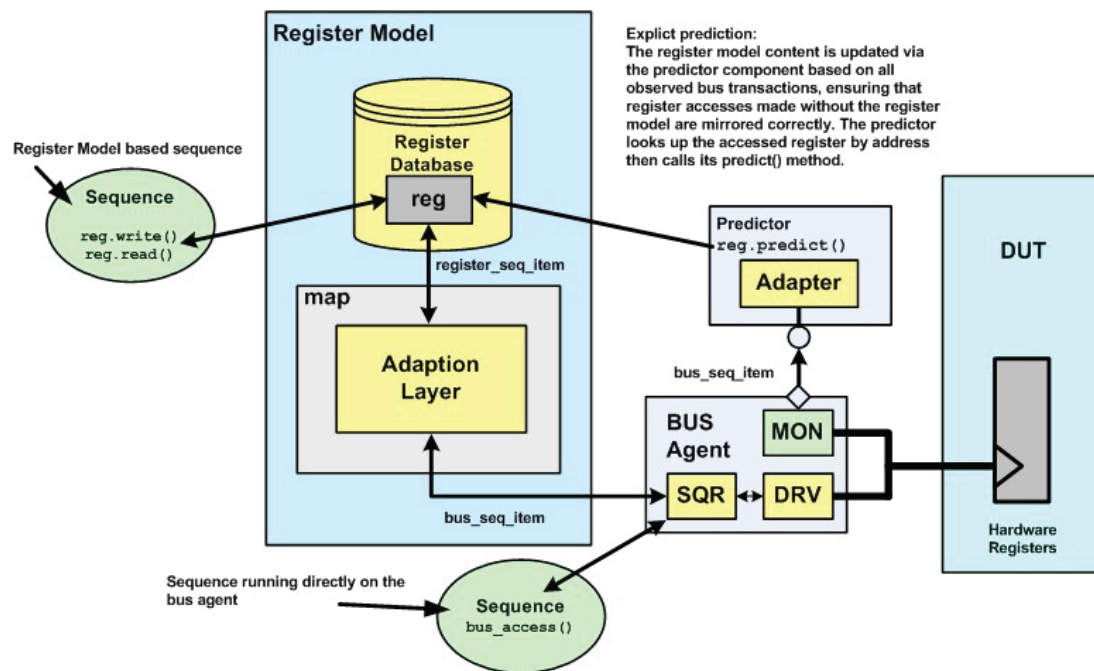
Integration Pre-requisites

If you are integrating a register model into a testbench, then the pre-requisites are that a register model has been written and that there is an adaptor class available for the bus agent that is going to be used to interact with the DUT bus interface.

Integration Process

In the testbench, the register model object needs to be constructed and a handle needs to be passed around the testbench environment using either the configuration and/or the resource mechanism.

In order to drive an agent from the register model an association needs to be made between it and the target sequencer so that when a sequence calls one of the register model methods a bus level sequence_item is sent to the target bus driver. The register model is kept updated with the current hardware register state via the bus agent monitor, and a predictor component is used to convert bus agent analysis transactions into updates of the register model.



Explicit Prediction Use Model

The testbench integrator might also be involved with implementing other analysis components which reference the register model, and these would include a scoreboard and a functional coverage monitor.

For the testbench integrator, the recommended route through the register material is outlined in the table below:

Step	Page	Description	Relevance
1	RegisterModelOverview	Overview of the register model hierarchy	Useful to understand terminology
2	Integrating	Overview of the register model stimulus and prediction architecture	Essential
3	Adapter	Adapter Implementation detail	Useful background
4	AdapterContext	How to provide context to an adapter, with an example	Useful background
5	integration	Register model integration detail	Essential
6	Scoreboarding	Scoreboard implementation	Useful background
7	FunctionalCoverage	Coverage implementation	Useful background

Using A Register Model

Once it has been integrated, the register model is used by the testbench user to create stimulus using sequences or through analysis components such as scoreboards and functional coverage monitors.

The register model is intended to make it easier to write reusable sequences that access hardware registers and areas of memory. The model data structure is organised to reflect the DUT hierarchy and this makes it easier to write abstract and reusable stimulus in terms of hardware blocks, memories, registers and fields rather than working at a lower bit pattern level of abstraction. The model contains a number of access methods which sequences use to read and write registers. These methods cause generic register transactions to be converted into transactions on the target bus.

The UVM package contains a library of built-in test sequences which can be used to do most of the basic register and memory tests, such as checking register reset values and checking the register and memory data paths. These tests can be disabled for those areas of the register or memory map where they are not relevant using register attributes.

One common form of stimulus is referred to as configuration. This is when a programmable DUT has its registers set up to support a particular mode of operation. The register model can support auto-configuration, a process whereby the contents of the register model are forced into a state that represents a device configuration using constrained randomization and then transferred into the DUT.

The register model supports front door and back door access to the DUT registers. Front door access uses the bus agent in the testbench and register accesses use the normal bus transfer protocol. Back door access uses simulator data base access routines to directly force or observe the register hardware bits in zero time, by-passing the normal bus interface logic.

As a verification environment evolves, users may well develop analysis components such as scoreboards and functional coverage monitors which refer to the contents of the register model in order to check DUT behaviour or to ensure that it has been tested in all required configurations.

If you are a testbench consumer using the register model, then you should read the following topics in the recommended order:

Step	Page	Description	Relevance
1	Specification	Register Specification	Background
2	RegisterModelOverview	Register Model Hierarchy Overview	Essential to understand the terminology
3	Integrating	Register Model Testbench architecture	Background
4	StimulusAbstraction	Stimulus Abstraction for registers	Essential
5	MemoryStimulus	Memory stimulus abstraction	Essential
6	BackdoorAccess	Back door accesses	Relevant if you need to do backdoor accesses
7	SequenceExamples	Example sequences	Essential
8	Configuration	How to configure a programmable DUT	Essential
9	BuiltInSequences	How to use the UVM built-in register sequences	May be relevant
10	Scoreboarding	Implementing a register model based scoreboard	Important if you need to maintain a scoreboard.
11	FunctionalCoverage	Implementing functional coverage using the register model	Important if you need to enhance a functional coverage model

Register Model Examples

The UVM register use model is illustrated by code excerpts which are taken from two example testbenches. The main example is a complete verification environment for a SPI master DUT, in addition to register model this includes a scoreboard and a functional coverage monitor, along with a number of test cases based on the use of register based sequences. The other example is designed to illustrate the use of memories and some of the built-in register sequences from the UVM library. Download links for these examples are provided in the table below:

Example	Download Link
SPI Master Testbench	
Memory Sub-System Testbench	

Register Model Overview

Introduction

In order to be able to use the UVM register model effectively, it is important to have a mental model of how it is structured in order to be able to find your way around it.

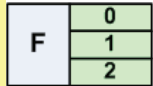
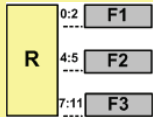
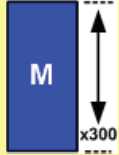
The register model is implemented using five main building blocks - the register field; the register; the memory; the register block; and the register map. The register field models a collection of bits that are associated with a function within a register. A field will have a width and a bit offset position within the register. A field can have different access modes such as read/write, read only or write only. A register contains one or more fields. A register block corresponds to a hardware block and contains one or more registers. A register block also contains one or more register maps.

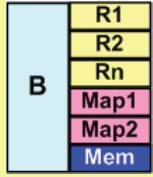
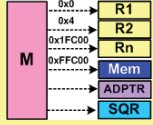
A memory region in the design is modeled by a `uvm_mem` which has a range, or size, and is contained within a register block and has an offset determined by a register map. A memory region is modeled as either read only, write only or read-write with all accesses using the full width of the data field. A `uvm_memory` does not contain fields.

The register map defines the address space offsets of one or more registers or memories in its parent block from the point of view of a specific bus interface. A group of registers may be accessible from another bus interface by a different set of address offsets and this can be modeled by using another address map within the parent block. The address map is also used to specify which bus agent is used when a register access takes place and which adapter is used to convert generic register transfers to/from target bus transfer sequence_items.

Register Model Layer Overview

The UVM register model is built up in layers:

Layer	Register base class	Description	Diagram
Fields	<code>uvm_reg_field</code>	Bit(s) grouped according to function within a register	
Register	<code>uvm_reg</code>	Collection of fields at different bit offset	
Memory	<code>uvm_mem</code>	Represents a block of memory which extends over a specified range	

Block	<i>uvm_block</i>	Collection of registers (Hardware block level), or sub-blocks (Sub-system level) with one or more maps. May also include memories.	
Map	<i>uvm_map</i>	Named address map which locates the offset address of registers, memories or sub-blocks. Also defines the target sequencer for register accesses from the map.	

Register Model Data Types

The register model uses some specific data types in order to standardize the width of the address and data fields:

Type	Default width	`define	Description
<code>uvm_reg_data_t</code>	64 bits	<code>`UVM_REG_DATA_WIDTH</code>	Used for register data fields (<code>uvm_reg</code> , <code>uvm_reg_field</code> , <code>uvm_mem</code>)
<code>uvm_reg_addr_t</code>	64 bits	<code>`UVM_REG_ADDR_WIDTH</code>	Used for register address variables

Both of these types are based on the SystemVerilog bit type and are therefore 2 state. By default, they are 64 bits wide, but the width of the each type is determined by a `define which can be overloaded by specifying a new define on the compilation command line.

```
#
# To change the width of the register uvm_data_t
# using Questa
#
vlog +incdir+$(UVM_HOME)/src +define+UVM_REG_DATA_WIDTH=24
$(UVM_HOME)/src/uvm_pkg.sv
```

Since this is a compilation variable it requires that you recompile the UVM package and it also has a global effect impacting all register models, components and sequences which use these data types. Therefore, although it is possible to change the default width of this variable, it is not recommended since it could have potential side effects.

Register Fields

The bottom layer is the field which corresponds to one or more bits within a register. Each field definition is an instantiation of the `uvm_reg_field` class. Fields are contained within an `uvm_reg` class and they are constructed and then configured using the `configure()` method:

```
//
// uvm_field configure method prototype
//
function void configure(uvm_reg      parent,    // The containing
register
                        int unsigned size,      // How many bits wide
                        int unsigned lsb pos,   // Bit offset within
the register
                        string        access,    // "RW", "RO", "WO"
etc
                        bit           volatile, // Volatile if bit is
```

```

updated by hardware
                                uvm_reg_data_t reset,    // The reset value
                                bit                    has_reset, // Whether the bit is
reset
                                bit                    is_rand,  // Whether the bit
can be randomized
                                bit                    individually_accessible); //
i.e. Totally contained within a byte lane

```

How the configure method is used is shown in the register code example.

When the field is created, it takes its name from the string passed to its create method which by convention is the same as the name of its handle.

Registers

Registers are modeled by extending the `uvm_reg` class which is a container for field objects. The overall characteristics of the register are defined in its constructor method:

```

//
// uvm_reg constructor prototype:
//
function new (string name="",    // Register name
              int unsigned n_bits, // Register width in bits
              int has_coverage);  // Coverage model supported by the
register

```

The register class contains a build method which is used to create and configure the fields. Note that this build method is not called by the UVM build phase, since the register is an `uvm_object` rather than an `uvm_component`. The following code example shows how the SPI master CTRL register model is put together.

```

//-----
// ctrl

//-----

class ctrl extends uvm_reg;
    `uvm_object_utils(ctrl)

    rand uvm_reg_field acs;
    rand uvm_reg_field ie;
    rand uvm_reg_field lsb;
    rand uvm_reg_field tx_neg;
    rand uvm_reg_field rx_neg;
    rand uvm_reg_field go_bsy;
    uvm_reg_field reserved;
    rand uvm_reg_field char_len;

//-----
// new

```


Memories

Memories are modeled by extending the `uvm_mem` class. The register model treats memories as regions, or memory address ranges where accesses can take place. Unlike registers, memory values are not stored because of the workstation memory overhead involved.

The range and access type of the memory is defined via its constructor:

```
//
// uvm_mem constructor prototype:
//
function new (string          name,          // Name of the memory
model
              longint unsigned size,          // The address range
              int unsigned      n bits,        // The width of the
memory in bits
              string           access = "RW", // Access - one of "RW"
or "RO"
              int              has_coverage = UVM_NO_COVERAGE); //
Functional coverage
```

An example of a memory class implementation:

```
// Memory array 1 - Size 32'h2000;
class mem_1_model extends uvm_mem;

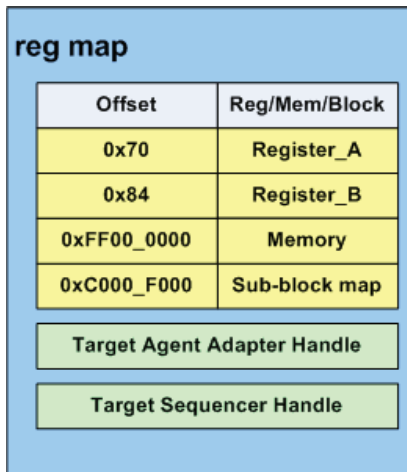
`uvm_object_utils(mem_1_model)

function new(string name = "mem_1_model");
    super.new(name, 32'h2000, 32, "RW", UVM_NO_COVERAGE);
endfunction

endclass: mem_1_model
```

Register Maps

The purpose of the register map is two fold. The map provides information on the offset of the registers, memories and/or register blocks contained within it. The map is also used to identify which bus agent register based sequences will be executed on, however this part of the register maps functionality is set up when integrating the register model into an UVM testbench.



Register Map Contents

In order to add a register or a memory to a map, the `add_reg()` or `add_mem()` methods are used. The prototypes for these methods are very similar:

```
//
// uvm_map add_reg method prototype:
//
function void add_reg (uvm_reg          rg,           // Register
object handle
                        uvm_reg_addr_t   offset,      // Register
address offset
                        string           rights = "RW", // Register
access policy
                        bit             unmapped=0,    // If true,
register does not appear in the address map
                                                // and a
frontdoor access needs to be defined
                        uvm_reg_frontdoor frontdoor=null); // Handle to
register frontdoor access object
//
// uvm_map add_mem method prototype:
//
function void add_mem (uvm_mem          mem,          // Memory
object handle
                        uvm_reg_addr_t offset,        // Memory
address offset
                        string           rights = "RW", // Memory
access policy
                        bit             unmapped=0,    // If true,
memory is not in the address map
```

```

// and a
frontdoor access needs to be defined
        uvm_reg_frontdoor frontdoor=null); // Handle to
memory frontdoor access object

```

There can be several register maps within a block, each one can specify a different address map and a different target bus agent.

Register Blocks

The next level of hierarchy in the UVM register structure is the `uvm_reg_block`. This class can be used as a container for registers and memories at the block level, representing the registers at the hardware functional block level, or as a container for multiple blocks representing the registers in a hardware sub-system or a complete SoC organized as blocks. In order to define register and memory address offsets the block contains an address map object derived from `uvm_reg_map`. A register map has to be created within the register block using the `create_map` method:

```

//
// Prototype for the create_map method
//
function uvm_reg_map create_map(string name,           // Name of
the map handle
                                uvm_reg_addr_t base_addr, // The maps
base address
                                int unsigned n_bytes,     // Map
access width in bytes
                                uvm_endianness_e endian,  // The
endianness of the map
                                bit byte_addressing=1);   // Whether
byte addressing is supported

//
// Example:
//
AHB map = create_map("AHB map", 'h0, 4, UVM LITTLE ENDIAN);

```

Note:

The `n_bytes` parameter is the word size (bus width) of the bus to which the map is associated. If a register's width exceeds the bus width, more than one bus access is needed to read and write that register over that bus. The `byte_addressing` argument affects how the address is incremented in these consecutive accesses. For example, if `n_bytes=4` and `byte_addressing=0`, then an access to a register that is 64-bits wide and at offset 0 will result in two bus accesses at addresses 0 and 1. With `byte_addressing=1`, that same access will result in two bus accesses at addresses 0 and 4.

The default for `byte_addressing` is 1.

The first map to be created within a register block is assigned to the `default_map` member of the register block.

The following code example is for the SPI master register block, this declares the register class handles for each of the registers in the SPI master, then the build method constructs and configures each of the registers before adding them to the `APB_map reg_map` at the appropriate offset address:

```
//-----
// spi_reg_block

//-----

class spi_reg_block extends uvm_reg_block;
  `uvm_object_utils(spi_reg_block)

  rand rtx0 rtx0_reg; rand
  rtx1 rtx1_reg; rand rtx2
  rtx2_reg; rand rtx3
  rtx3_reg; rand ctrl ctrl_reg;
  rand divider divider_reg;
  rand ss ss_reg;

  uvm_reg_map APB_map; // Block map

//-----

// new

//-----

function new(string name = "spi_reg_block");
  super.new(name, UVM_NO_COVERAGE);
endfunction

//-----

// build

//-----

virtual function void build();
  rtx0_reg = rtx0::type_id::create("rtx0");
  rtx0_reg.configure(this, null, ""); rtx0_reg.build();

  rtx1_reg = rtx1::type_id::create("rtx1");
  rtx1_reg.configure(this, null, ""); rtx1_reg.build();

  rtx2_reg = rtx2::type_id::create("rtx2");
  rtx2_reg.configure(this, null, ""); rtx2_reg.build();

  rtx3_reg = rtx3::type_id::create("rtx3");
  rtx3_reg.configure(this, null, ""); rtx3_reg.build();
endfunction

```

```

ctrl_reg = ctrl::type_id::create("ctrl");
ctrl_reg.configure(this, null, "");
ctrl_reg.build();

divider_reg = divider::type_id::create("divider");
divider_reg.configure(this, null, "");
divider_reg.build();

ss_reg = ss::type_id::create("ss");
ss_reg.configure(this, null, "");
ss_reg.build();

// Map name, Offset, Number of bytes, Endianess)
APB_map = create_map("APB_map", 'h0, 4, UVM_LITTLE_ENDIAN);

APB_map.add_reg(rxtx0_reg, 32'h00000000, "RW");
APB_map.add_reg(rxtx1_reg, 32'h00000004, "RW");
APB_map.add_reg(rxtx2_reg, 32'h00000008, "RW");
APB_map.add_reg(rxtx3_reg, 32'h0000000c, "RW");
APB_map.add_reg(ctrl_reg, 32'h00000010, "RW");
APB_map.add_reg(divider_reg, 32'h00000014, "RW");
APB_map.add_reg(ss_reg, 32'h00000018, "RW");

lock_model();
endfunction

endclass

```

Note that the final statement in the build method is the lock_model() method. This is used to finalize the address mapping and to ensure that the model cannot be altered by another user.

Hierarchical Register Blocks

The register block in the previous example can be used for block level verification, but if the SPI is integrated into a larger design, the SPI register block can be combined with other register blocks in an integration level block to create a new register model. The cluster block incorporates each sub-block and adds them to a new cluster level address map. This process can be repeated and a full SoC register map might contain several nested layers of register blocks. The following code example shows how this would be done for a sub-system containing the SPI master and a number of other peripheral blocks.

```

package pss_reg_pkg;

import uvm_pkg::*;
`include "uvm_macros.svh"

import spi_reg_pkg::*;
import gpio_reg_pkg::*;

```

```
class pss_reg_block extends uvm_reg_block;

`uvm_object_utils(pss_reg_block)

function new(string name = "pss_reg_block");
    super.new(name);
endfunction

rand spi_reg_block spi;
rand gpio_reg_block gpio;

function void build();
    AHB_map = create_map("AHB_map", 0, 4, UVM_LITTLE_ENDIAN);

    spi = spi_reg_block::type_id::create("spi");
    spi.configure(this);
    spi.build();
    AHB_map.add_submap(this.spi.default_map, 0);

    gpio = gpio_reg_block::type_id::create("gpio");
    gpio.configure(this);
    gpio.build();
    AHB_map.add_submap(this.gpio.default_map, 32'h100);

    lock_model();
endfunction: build

endclass: pss_reg_block

endpackage: pss_reg_pkg
```

Register Model Structure

Introduction

In order to be able to use the UVM register model effectively, it is important to have a mental model of how it is structured in order to be able to find your way around it.

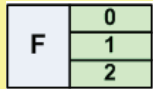
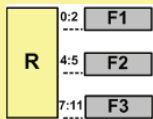
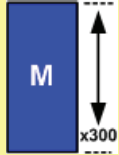
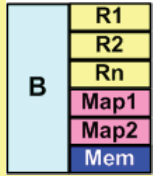
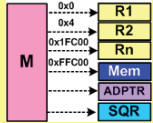
The register model is implemented using five main building blocks - the register field; the register; the memory; the register block; and the register map. The register field models a collection of bits that are associated with a function within a register. A field will have a width and a bit offset position within the register. A field can have different access modes such as read/write, read only or write only. A register contains one or more fields. A register block corresponds to a hardware block and contains one or more registers. A register block also contains one or more register maps.

A memory region in the design is modeled by a `uvm_mem` which has a range, or size, and is contained within a register block and has an offset determined by a register map. A memory region is modeled as either read only, write only or read-write with all accesses using the full width of the data field. A `uvm_memory` does not contain fields.

The register map defines the address space offsets of one or more registers or memories in its parent block from the point of view of a specific bus interface. A group of registers may be accessible from another bus interface by a different set of address offsets and this can be modeled by using another address map within the parent block. The address map is also used to specify which bus agent is used when a register access takes place and which adapter is used to convert generic register transfers to/from target bus transfer sequence_items.

Register Model Layer Overview

The UVM register model is built up in layers:

Layer	Register base class	Description	Diagram
Fields	<code>uvm_reg_field</code>	Bit(s) grouped according to function within a register	
Register	<code>uvm_reg</code>	Collection of fields at different bit offset	
Memory	<code>uvm_mem</code>	Represents a block of memory which extends over a specified range	
Block	<code>uvm_block</code>	Collection of registers (Hardware block level), or sub-blocks (Sub-system level) with one or more maps. May also include memories.	
Map	<code>uvm_map</code>	Named address map which locates the offset address of registers, memories or sub-blocks. Also defines the target sequencer for register accesses from the map.	

Register Model Data Types

The register model uses some specific data types in order to standardize the width of the address and data fields:

Type	Default width	`define	Description
uvm_reg_data_t	64 bits	`UVM_REG_DATA_WIDTH	Used for register data fields (uvm_reg, uvm_reg_field, uvm_mem)
uvm_reg_addr_t	64 bits	`UVM_REG_ADDR_WIDTH	Used for register address variables

Both of these types are based on the SystemVerilog bit type and are therefore 2 state. By default, they are 64 bits wide, but the width of the each type is determined by a `define which can be overloaded by specifying a new define on the compilation command line.

```
#
# To change the width of the register uvm_data_t
# using Questa
#
vlog +incdir+$(UVM_HOME)/src +define+UVM_REG_DATA_WIDTH=24
$(UVM_HOME)/src/uvm_pkg.sv
```

Since this is a compilation variable it requires that you recompile the UVM package and it also has a global effect impacting all register models, components and sequences which use these data types. Therefore, although it is possible to change the default width of this variable, it is not recommended since it could have potential side effects.

Register Fields

The bottom layer is the field which corresponds to one or more bits within a register. Each field definition is an instantiation of the uvm_reg_field class. Fields are contained within an uvm_reg class and they are constructed and then configured using the configure() method:

```
//
// uvm_field configure method prototype
//
function void configure(uvm_reg      parent,      // The containing
register                                register
                                int unsigned size,    // How many bits wide
                                int unsigned lsb pos,  // Bit offset within
the register
                                string      access,    // "RW", "RO", "WO"
etc
                                bit          volatile, // Volatile if bit is
updated by hardware
                                uvm_reg_data_t reset,   // The reset value
                                bit          has reset, // Whether the bit is
reset
                                bit          is rand,   // Whether the bit
can be randomized
                                bit          individually accessible); //
i.e. Totally contained within a byte lane
```

How the configure method is used is shown in the register code example.

When the field is created, it takes its name from the string passed to its create method which by convention is the same as the name of its handle.

Registers

Registers are modeled by extending the `uvm_reg` class which is a container for field objects. The overall characteristics of the register are defined in its constructor method:

```
//
// uvm_reg constructor prototype:
//
function new (string name="",      // Register name
             int unsigned n_bits, // Register width in bits
             int has_coverage);    // Coverage model supported by the
register
```

The register class contains a build method which is used to create and configure the fields. Note that this build method is not called by the UVM build phase, since the register is an `uvm_object` rather than an `uvm_component`. The following code example shows how the SPI master CTRL register model is put together.

```
//-----
// ctrl

//-----

class ctrl extends uvm_reg;
    `uvm_object_utils(ctrl)

    rand uvm_reg_field acs;
    rand uvm_reg_field ie;
    rand uvm_reg_field lsb;
    rand uvm_reg_field tx_neg;
    rand uvm_reg_field rx_neg;
    rand uvm_reg_field go_bsy;
    uvm_reg_field reserved;
    rand uvm_reg_field char_len;

//-----

// new

//-----

function new(string name = "ctrl");
    super.new(name, 14, UVM_NO_COVERAGE);
endfunction

//-----

// build

//-----
```

```

virtual function void build();

    acs = uvm_reg_field::type_id::create("acs"); ie =
    uvm_reg_field::type_id::create("ie"); lsb =
    uvm_reg_field::type_id::create("lsb");
    tx_neg = uvm_reg_field::type_id::create("tx_neg"); rx_neg =
    uvm_reg_field::type_id::create("rx_neg"); go_bsy =
    uvm_reg_field::type_id::create("go_bsy"); reserved =
    uvm_reg_field::type_id::create("reserved"); char_len =
    uvm_reg_field::type_id::create("char_len");

    acs.configure(this, 1, 13, "RW", 0, 1'b0, 1, 1, 0);
    ie.configure(this, 1, 12, "RW", 0, 1'b0, 1, 1, 0);
    lsb.configure(this, 1, 11, "RW", 0, 1'b0, 1, 1, 0);
    tx_neg.configure(this, 1, 10, "RW", 0, 1'b0, 1, 1, 0);
    rx_neg.configure(this, 1, 9, "RW", 0, 1'b0, 1, 1, 0);
    go_bsy.configure(this, 1, 8, "RW", 0, 1'b0, 1, 1, 0);
    reserved.configure(this, 1, 7, "RO", 0, 1'b0, 1, 0, 0);
    char_len.configure(this, 7, 0, "RW", 0, 7'b0000000, 1, 1, 0);

endfunction
endclass

```

When a register is added to a block it is created, causing its fields to be created and configured, and then it is configured before it is added to one or more reg_maps to define its memory offset. The prototype for the register configure() method is as follows:

```

//
// Register configure method prototype
//
function void configure (uvm_reg_block blk_parent,           // The
containing reg block
                                uvm_reg_file regfile_parent = null, //
Optional, not used
                                string hdl_path = "");         // Used if
    HW register can be specified in one
                                                                //
hdl path string

```

Memories

Memories are modeled by extending the uvm_mem class. The register model treats memories as regions, or memory address ranges where accesses can take place. Unlike registers, memory values are not stored because of the workstation memory overhead involved.

The range and access type of the memory is defined via its constructor:

```

//
// uvm_mem constructor prototype:
//
function new (string          name,           // Name of the memory

```

```

model
    longint unsigned size,          // The address range
    int unsigned      n bits,      // The width of the
memory in bits
    string            access = "RW", // Access - one of "RW"
or "RO"
    int                has_coverage = UVM_NO_COVERAGE); //
Functional coverage

```

An example of a memory class implementation:

```

// Memory array 1 - Size 32'h2000;
class mem_1_model extends uvm_mem;

`uvm_object_utils(mem_1_model)

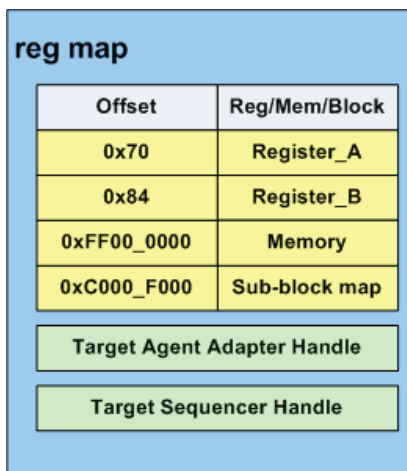
function new(string name = "mem_1_model");
    super.new(name, 32'h2000, 32, "RW", UVM_NO_COVERAGE);
endfunction

endclass: mem_1_model

```

Register Maps

The purpose of the register map is two fold. The map provides information on the offset of the registers, memories and/or register blocks contained within it. The map is also used to identify which bus agent register based sequences will be executed on, however this part of the register maps functionality is set up when integrating the register model into an UVM testbench.



Register Map Contents

In order to add a register or a memory to a map, the `add_reg()` or `add_mem()` methods are used. The prototypes for these methods are very similar:

```

//
// uvm_map add_reg method prototype:
//
function void add_reg (uvm_reg          rg,          // Register
object handle

```

```

        uvm_reg_addr_t offset,          // Register
address offset

        string rights = "RW",          // Register
access policy

        bit unmapped=0,                // If true,
register does not appear in the address map
                                         // and a
frontdoor access needs to be defined
        uvm_reg_frontdoor frontdoor=null); // Handle to
register frontdoor access object
//
// uvm_map add_mem method prototype:
//
function void add_mem (uvm_mem mem,          // Memory
object handle
        uvm_reg_addr_t offset,          // Memory
address offset
        string rights = "RW",          // Memory
access policy
        bit unmapped=0,                // If true,
memory is not in the address map
                                         // and a
frontdoor access needs to be defined
        uvm_reg_frontdoor frontdoor=null); // Handle to
memory frontdoor access object

```

There can be several register maps within a block, each one can specify a different address map and a different target bus agent.

Register Blocks

The next level of hierarchy in the UVM register structure is the `uvm_reg_block`. This class can be used as a container for registers and memories at the block level, representing the registers at the hardware functional block level, or as a container for multiple blocks representing the registers in a hardware sub-system or a complete SoC organized as blocks. In order to define register and memory address offsets the block contains an address map object derived from `uvm_reg_map`. A register map has to be created within the register block using the `create_map` method:

```

//
// Prototype for the create_map method
//
function uvm_reg_map create_map(string name,          // Name of
the map handle
        uvm_reg_addr_t base_addr,          // The maps
base address
        int unsigned n_bytes,            // Map
access width in bytes
        uvm_endianness_e endian,          // The
endianness of the map
        bit byte_addressing=1);          // Whether

```

```
byte_addressing is supported

//
// Example:
//
AHB_map = create_map("AHB_map", 'h0, 4, UVM_LITTLE_ENDIAN);
```

Note:

The *n_bytes* parameter is the word size (bus width) of the bus to which the map is associated. If a register's width exceeds the bus width, more than one bus access is needed to read and write that register over that bus. The *byte_addressing* argument affects how the address is incremented in these consecutive accesses. For example, if *n_bytes*=4 and *byte_addressing*=0, then an access to a register that is 64-bits wide and at offset 0 will result in two bus accesses at addresses 0 and 1. With *byte_addressing*=1, that same access will result in two bus accesses at addresses 0 and 4.

The default for *byte_addressing* is 1.

The first map to be created within a register block is assigned to the *default_map* member of the register block.

The following code example is for the SPI master register block, this declares the register class handles for each of the registers in the SPI master, then the build method constructs and configures each of the registers before adding them to the APB_map *reg_map* at the appropriate offset address:

```
//-----
// spi_reg_block

//-----

class spi_reg_block extends uvm_reg_block;
    `uvm_object_utils(spi_reg_block)

    rand rxtx0 rxtx0_reg;
    rand rxtx1 rxtx1_reg;
    rand rxtx2 rxtx2_reg;
    rand rxtx3 rxtx3_reg;
    rand ctrl ctrl_reg;
    rand divider divider_reg;
    rand ss ss_reg;

    uvm_reg_map APB_map; // Block map

//-----

// new

//-----

function new(string name = "spi_reg_block");
    super.new(name, UVM_NO_COVERAGE);
endfunction
```

```

//-----
// build

//-----

virtual function void build();
    rtx0_reg = rtx0::type_id::create("rtx0");
    rtx0_reg.configure(this, null, ""); rtx0_reg.build();

    rtx1_reg = rtx1::type_id::create("rtx1");
    rtx1_reg.configure(this, null, ""); rtx1_reg.build();

    rtx2_reg = rtx2::type_id::create("rtx2");
    rtx2_reg.configure(this, null, ""); rtx2_reg.build();

    rtx3_reg = rtx3::type_id::create("rtx3");
    rtx3_reg.configure(this, null, ""); rtx3_reg.build();

    ctrl_reg = ctrl::type_id::create("ctrl"); ctrl_reg.configure(this,
null, ""); ctrl_reg.build();

    divider_reg = divider::type_id::create("divider");
    divider_reg.configure(this, null, ""); divider_reg.build();

    ss_reg = ss::type_id::create("ss");
    ss_reg.configure(this, null, ""); ss_reg.build();

    // Map name, Offset, Number of bytes, Endianess)
    APB_map = create_map("APB_map", 'h0, 4, UVM_LITTLE_ENDIAN);

    APB_map.add_reg(rtx0_reg, 32'h00000000, "RW");
    APB_map.add_reg(rtx1_reg, 32'h00000004, "RW");
    APB_map.add_reg(rtx2_reg, 32'h00000008, "RW");
    APB_map.add_reg(rtx3_reg, 32'h0000000c, "RW");
    APB_map.add_reg(ctrl_reg, 32'h00000010, "RW");
    APB_map.add_reg(divider_reg, 32'h00000014, "RW");
    APB_map.add_reg(ss_reg, 32'h00000018, "RW");

    lock_model();
endfunction

```

```
endclass
```

Note that the final statement in the build method is the lock_model() method. This is used to finalize the address mapping and to ensure that the model cannot be altered by another user.

Hierarchical Register Blocks

The register block in the previous example can be used for block level verification, but if the SPI is integrated into a larger design, the SPI register block can be combined with other register blocks in an integration level block to create a new register model. The cluster block incorporates each sub-block and adds them to a new cluster level address map. This process can be repeated and a full SoC register map might contain several nested layers of register blocks. The following code example shows how this would be done for a sub-system containing the SPI master and a number of other peripheral blocks.

```
package pss_reg_pkg;

import uvm_pkg::*;
`include "uvm_macros.svh"

import spi_reg_pkg::*;
import gpio_reg_pkg::*;

class pss_reg_block extends uvm_reg_block;

  `uvm_object_utils(pss_reg_block)

  function new(string name = "pss_reg_block");
    super.new(name);
  endfunction

  rand spi_reg_block spi;
  rand gpio_reg_block gpio;

  function void build();
    AHB_map = create_map("AHB_map", 0, 4, UVM_LITTLE_ENDIAN);

    spi = spi_reg_block::type_id::create("spi");
    spi.configure(this);
    spi.build();
    AHB_map.add_submap(this.spi.default_map, 0);

    gpio = gpio_reg_block::type_id::create("gpio");
    gpio.configure(this);
    gpio.build();
    AHB_map.add_submap(this.gpio.default_map, 32'h100);

    lock_model();
  endfunction: build
```



```

endclass: pss_reg_block

endpackage: pss_reg_pkg

```

Complex Address Maps

In SoC design, the address mapping of registers and memory is often more complex than a single map. When there are several masters in a system, then they often have a different view of the register map, i.e. the same register will appear at different addresses depending on which bus master is accessing it. Another common scenario is for the address map to be dynamic.

Multiple Address Mapping

If the hardware register space can be accessed by more than one bus interface, then the block can contain multiple address maps to support alternative address maps. In the following example, two maps are created and have memories and registers added to them at different offsets:

```

//
// Memory sub-system (mem_ss) register & memory block
//
class mem_ss_reg_block extends uvm_reg_block;

  `uvm_object_utils(mem_ss_reg_block)

  function new(string name = "mem_ss_reg_block");
    super.new(name, build_coverage(UVM_CVR_ADDR_MAP));
  endfunction

  // Mem array configuration registers
  rand mem_offset_reg mem_1_offset;
  rand mem_range_reg mem_1_range;
  rand mem_offset_reg mem_2_offset;
  rand mem_range_reg mem_2_range;
  rand mem_offset_reg mem_3_offset;
  rand mem_range_reg mem_3_range;
  rand mem_status_reg mem_status;

  // Memories
  rand mem_1_model mem_1;
  rand mem_2_model mem_2;
  rand mem_3_model mem_3;

  // Map
  uvm_reg_map AHB_map;
  uvm_reg_map AHB_2_map;

```

```

function void build();
    mem_1_offset = mem_offset_reg::type_id::create("mem_1_offset");
    mem_1_offset.configure(this, null, "");
    mem_1_offset.build();
    mem_1_range = mem_range_reg::type_id::create("mem_1_range");
    mem_1_range.configure(this, null, "");
    mem_1_range.build();
    mem_2_offset = mem_offset_reg::type_id::create("mem_2_offset");
    mem_2_offset.configure(this, null, "");
    mem_2_offset.build();
    mem_2_range = mem_range_reg::type_id::create("mem_2_range");
    mem_2_range.configure(this, null, "");
    mem_2_range.build();
    mem_3_offset = mem_offset_reg::type_id::create("mem_3_offset");
    mem_3_offset.configure(this, null, "");
    mem_3_offset.build();
    mem_3_range = mem_range_reg::type_id::create("mem_3_range");
    mem_3_range.configure(this, null, "");
    mem_3_range.build();
    mem_status = mem_status_reg::type_id::create("mem_status");
    mem_status.configure(this, null, "");
    mem_status.build();
    mem_1 = mem_1_model::type_id::create("mem_1");
    mem_1.configure(this, "");
    mem_2 = mem_2_model::type_id::create("mem_2");
    mem_2.configure(this, "");
    mem_3 = mem_3_model::type_id::create("mem_3");
    mem_3.configure(this, "");
    // Create the maps
    AHB_map = create_map("AHB_map", 'h0, 4, UVM_LITTLE_ENDIAN, 1); AHB_2_map =
    create_map("AHB_2_map", 'h0, 4, UVM_LITTLE_ENDIAN, 1);
    // Add registers and memories to the AHB_map
    AHB_map.add_reg(mem_1_offset, 32'h00000000, "RW");
    AHB_map.add_reg(mem_1_range, 32'h00000004, "RW");
    AHB_map.add_reg(mem_2_offset, 32'h00000008, "RW");
    AHB_map.add_reg(mem_2_range, 32'h0000000c, "RW");
    AHB_map.add_reg(mem_3_offset, 32'h00000010, "RW");
    AHB_map.add_reg(mem_3_range, 32'h00000014, "RW");
    AHB_map.add_reg(mem_status, 32'h00000018, "RO");
    AHB_map.add_mem(mem_1, 32'hF000_0000, "RW");
    AHB_map.add_mem(mem_2, 32'hA000_0000, "RW");
    AHB_map.add_mem(mem_3, 32'h0001_0000, "RW");
    // Add registers and memories to the AHB_2_map
    AHB_2_map.add_reg(mem_1_offset, 32'h8000_0000, "RW");
    AHB_2_map.add_reg(mem_1_range, 32'h8000_0004, "RW");
    AHB_2_map.add_reg(mem_2_offset, 32'h8000_0008, "RW");
    AHB_2_map.add_reg(mem_2_range, 32'h8000_000c, "RW");

```

```

    AHB_2_map.add_reg(mem_3_offset, 32'h8000_0010, "RW");
    AHB_2_map.add_reg(mem_3_range, 32'h8000_0014, "RW");
    AHB_2_map.add_reg(mem_status, 32'h8000_0018, "RO");
    AHB_2_map.add_mem(mem_1, 32'h7000_0000, "RW");
    AHB_2_map.add_mem(mem_2, 32'h2000_0000, "RW");
    AHB_2_map.add_mem(mem_3, 32'h0001_0000, "RW");

    lock_model();
endfunction: build

endclass: mem_ss_reg_block

```

Dynamic Address Mapping

In SoC designs, address maps can be changed on the fly for reasons including the following:

- Security - Registers are made inaccessible in different states
- Reconfiguration - After discovery or re-enabling of a hardware resource, the address map has to be reconfigured
- An additional host is added to the system, requiring a new address view to be added to the address map
- Virtualisation - Where during an interrupt different virtual machines have to take over common resources

The UVM register map supports dynamic re-mapping by allowing the user to unlock and reconfigure a register block. After the `unlock_model()` operation, the `unregister()` function can be used to remove a register map from the block, additional register maps can be added to the block, and map register content can be unregistered or added to.

Since a SoC register model may have to support several different register map configurations that could change dynamically during a test, the SoC register block should contain re-map functions to make the operation easy to use.

Dynamically Removing/Re-Adding Registers and Memories

Assuming that we are using the `mem_ss_reg_block` defined previously, there might be a chip level security mode where certain registers or memory regions would not be available to users. The following example shows an example method for going into a secure mode by unmapping `mem3` and its associated registers, and then another for exiting the secure mode by re-instating the mapping of `mem3` and its registers.

```

//
// Memory sub-system (mem_ss) register & memory block
//
class mem_ss_reg_block extends uvm_reg_block;

    `uvm_object_utils(mem_ss_reg_block)

    function new(string name = "mem_ss_reg_block");
        super.new(name, build_coverage(UVM_CVR_ADDR_MAP));
    endfunction

    // Removes mem3 and the registers associated with it from the AHB_map
    function void security_demap();
        unlock_model();
        AHB_map.unregister(mem3_offset);
        AHB_map.unregister(mem3_range);
    endfunction
endclass

```

```

    AHB_map.unregister(mem3);
    lock_model();
endfunction

// Re-instates mem3 and the registers associated with it into the
AHB_map
function void security_remap();
    unlock_model();
    AHB_map.add_reg(mem_3_offset, 32'h00000010, "RW");
    AHB_map.add_reg(mem_3_range, 32'h00000014, "RW");
    AHB_map.add_mem(mem_3, 32'h0001_0000, "RW");
    lock_model();
endfunction

endclass: mem_ss_reg_block

```

Dynamically Changing An Address Map

The following code example illustrates a method for changing an address map to a new mapping, and another method to re-mapped it back to its original configuration. Again, it is based on adding methods to the `mem_ss_reg_block`.

```

//
// Memory sub-system (mem_ss) register & memory block
//
class mem_ss_reg_block extends uvm_reg_block;

// Remaps the contents of the AHB_map for host_B
function void remap_for_host_B();
    unlock_model();
    unregister(AHB_map); // Unregisters the address map
    AHB_map = null;      // Delete the original AHB_map
    // Create and configure the new version of the AHB_map
    AHB_map = create_map("AHB_map", 'h0, 4, UVM_LITTLE_ENDIAN, 1);
    // Add registers and memories to the AHB_map
    AHB_map.add_reg(mem_1_offset, 32'h20000000, "RW");
    AHB_map.add_reg(mem_1_range, 32'h20000004, "RW");
    AHB_map.add_reg(mem_2_offset, 32'h20000008, "RW");
    AHB_map.add_reg(mem_2_range, 32'h2000000c, "RW");
    AHB_map.add_reg(mem_3_offset, 32'h20000010, "RW");
    AHB_map.add_reg(mem_3_range, 32'h20000014, "RW");
    AHB_map.add_reg(mem_status, 32'h20000018, "RO");
    AHB_map.add_mem(mem_1, 32'hE000_0000, "RW");
    AHB_map.add_mem(mem_2, 32'hC000_0000, "RW");
    AHB_map.add_mem(mem_3, 32'h4000_0000, "RW");
    lock_model();
endfunction

// Re-instates the default AHB_map for host_A

```

```

function void remap_to_default();
    unlock_model();
    unregister(AHB_map);
    AHB_map = null;
    AHB_map = create_map("AHB_map", 'h0, 4, UVM_LITTLE_ENDIAN, 1);
    // Add registers and memories to the AHB_map
    AHB_map.add_reg(mem_1_offset, 32'h00000000, "RW");
    AHB_map.add_reg(mem_1_range, 32'h00000004, "RW");
    AHB_map.add_reg(mem_2_offset, 32'h00000008, "RW");
    AHB_map.add_reg(mem_2_range, 32'h0000000c, "RW");
    AHB_map.add_reg(mem_3_offset, 32'h00000010, "RW");
    AHB_map.add_reg(mem_3_range, 32'h00000014, "RW");
    AHB_map.add_reg(mem_status, 32'h00000018, "RO");
    AHB_map.add_mem(mem_1, 32'hF000_0000, "RW");
    AHB_map.add_mem(mem_2, 32'hA000_0000, "RW");
    AHB_map.add_mem(mem_3, 32'h0001_0000, "RW");
    lock_model();
endfunction

endclass: mem_ss_reg_block

```

Specifying Registers

Hardware functional blocks connected to host processors are managed via memory mapped registers. This means that each bit in the software address map corresponds to a hardware flip-flop. In order to control and interact with the hardware, software has to read and write the registers and so the register description is organized using an abstraction which is referred to as the hardware-software interface, or as the register description.

This hardware-software interface allocates addresses in the I/O memory map to a register which is identified by a mnemonic. Each register may then be broken down into fields, or groups of individual bits which again are given a mnemonic. A field may just be a single bit, or it may be as wide as the register itself. Each field may have different access attributes, for instance it might be read-only, write-only, read-write or clear on read. The register may have reserved fields, in other words bits in the register which are not used but might be used in future versions of the design. Here is an example of a register description - the control register from an SPI master DUT.

SPI Control Register - Reset Value = 0

Bit Pos	31:14	13	12	11	10	9	8	7	6:0
Access	RO	R/W	R/W	R/W	R/W	R/W	R/W	RO	R/W
Name	Reserved	ACS	IE	LSB	Tx_NEG	Rx_NEG	GO_BSY	Reserved	CHAR_LEN

For a given functional block, there may be multiple registers and each one will have an address which is offset from the base address of the block. To access a register, a processor will do a read or write to its address. The software engineer uses the register map to determine how to program a hardware device, and he may well use a set of define statements which map register names and field names to numeric values in a header file so that he can work at a more abstract level of detail. For instance, if he wishes to enable interrupts in the SPI master he may do a read from

CTRL, then OR the value with IE and then write back the resultant value to CTRL:

```
spi_ctrl = reg_read(CTRL);
spi_ctrl = spi_ctrl | IE;
reg_write(CTRL, spi_ctrl);
```

The register address map for the SPI master is as in the table below. Note that the control register is at an offset address of 0x10 from the SPI master base address.

SPI Master Register Address Map

Name	Address Offset	Width	Access	Description
RX0	0x00	32	RO	Receive data register 0
TX0	0x00	32	WO	Transmit data register 0
RX1	0x04	32	RO	Receive data register 1
TX1	0x04	32	WO	Transmit data register 1
RX2	0x08	32	RO	Receive data register 2
TX2	0x08	32	WO	Transmit data register 2
RX3	0x0c	32	RO	Receive data register 3
TX3	0x0c	32	WO	Transmit data register 3
CTRL	0x10	32	R/W	Control and status register
DIVIDER	0x14	32	R/W	Clock divider register
SS	0x18	32	R/W	Slave Select Register

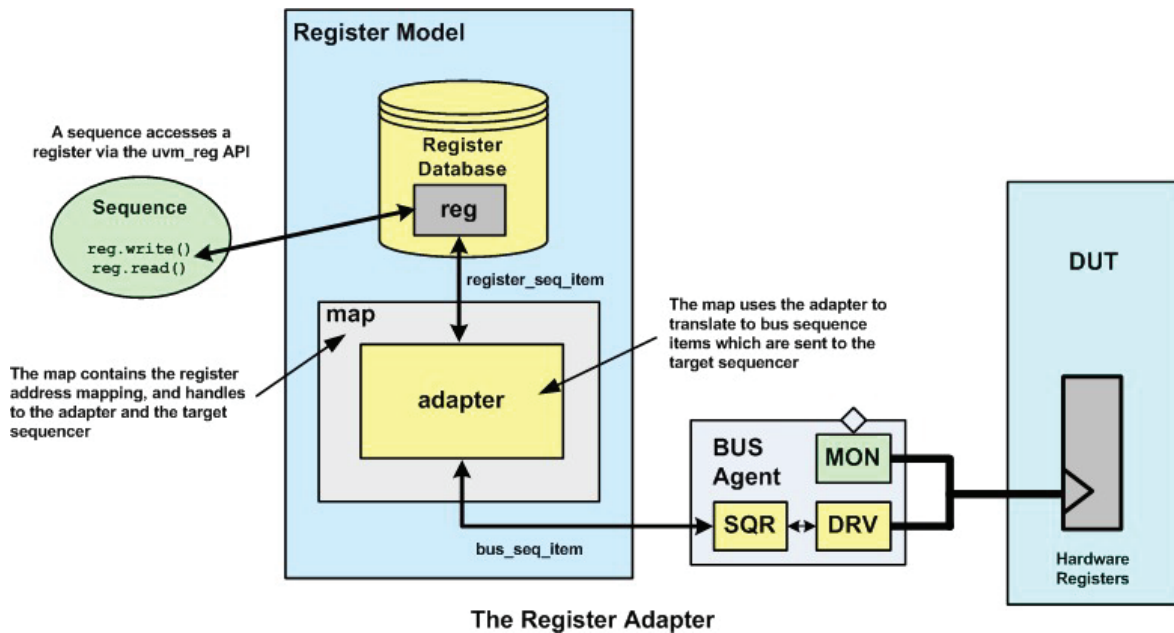
Note: In the SPI master the RXn and TXn registers actually map onto the same hardware flip-flops and during character transmission and reception their content changes. Therefore from the software interface abstraction the TX registers are write only, and the RX registers are read only, and the Rx content is only valid at the end of a character transfer.

If the SPI master is integrated into a larger design then the base address of the SPI master will change, the software engineer protects himself from the effects of that change by updating his header file, but the underlying firmware code does not need to change.

The Register Layer Adapter

Overview

The UVM register model access methods generate bus read and write cycles using generic register transactions. These transactions need to be adapted to the target bus sequence_item. The adapter needs to be bidirectional in order to convert register transaction requests to bus sequence items, and to be able to convert bus sequence item responses back to register sequence items. The adapter should be implemented by extending the `uvm_reg_adapter` base class.



Adapter Is Part Of An Agent Package

The register adapter should be supplied as part of the target bus agent package, but since the UVM register model capability is relatively new you will most likely have to implement your own register adapter until the agent provider incorporates one into the agent package.

If you are creating your own bus agent, then you should include an adapter to allow users of your agent package to be able to use it with the register model.

Implementing An Adapter

The generic register item is implemented as a struct in order to minimise the amount of memory resource it uses. The struct is defined as type `uvm_reg_bus_op` and this contains 6 fields:

Property	Type	Comment/Description
addr	uvm_reg_addr_t	Address field, defaults to 64 bits
data	uvm_reg_data_t	Read or write data, defaults to 64 bits
kind	uvm_access_e	UVM_READ or UVM_WRITE
n_bits	unsigned int	Number of bits being transferred
byte_en	uvm_reg_byte_en_t	Byte enable
status	uvm_status_e	UVM_IS_OK, UVM_IS_X, UVM_NOT_OK

These fields need to be mapped to/from the target bus sequence item and this is done by extending the `uvm_reg_adapter` class which contains two methods - `reg2bus()` and `bus2reg()` which need to be overlaid. The adapter class also contains two property bits - `supports_byte_enable` and `provides_responses`, these should be set according to the functionality supported by the target bus and the target bus agent.

uvm_reg_adapter	
Methods	Description
reg2bus	Overload to convert generic register access items to target bus agent sequence items
bus2reg	Overload to convert target bus sequence items to register model items
Properties (Of type bit)	Description
supports_byte_enable	Set to 1 if the target bus and the target bus agent supports byte enables, else set to 0
provides_responses	Set to 1 if the target agent driver sends separate response sequence_items that require response handling

Taking the APB bus as a simple example; the bus sequence_item, `apb_seq_item`, contains 3 fields (`addr`, `data`, `we`) which correspond to address, data, and bus direction. Address and data map directly and the APB item write enable maps to the bus item kind field. When converting an APB bus response to a register item the status field will be set to `UVM_IS_OK` since the APB agent does not support the `SLVERR` status bit used by the APB.

Since the APB bus does not support byte enables, the `supports_byte_enable` bit is set to 0 in the constructor of the APB adapter.

The `provides_responses` bit should be set if the agent driver returns a separate response item (i.e. `put(response)`, or `item_done(response)`) from its request item - see Driver/Sequence API. This bit is used by the register model layering code to determine whether to wait for a response or not, if it is set and the driver does not return a response, then the stimulus generation will lock-up.

Since the APB driver being used returns an `item_done()`, it therefore uses a single item for both request and response, so the `provides_responses` bit is also set to 0 in the constructor.

The example code for the register to APB adapter is as follows:

```
class reg2apb_adapter extends uvm_reg_adapter;

  `uvm_object_utils(reg2apb_adapter)

  function new(string name = "reg2apb_adapter");
    super.new(name);
    supports_byte_enable = 0;
    provides_responses = 0;
  endfunction
```



```

virtual function uvm_sequence_item reg2bus(const ref uvm_reg_bus_op
rw);
    apb_seq_item apb = apb_seq_item::type_id::create("apb");
    apb.we = (rw.kind == UVM_READ) ? 0 : 1;
    apb.addr = rw.addr;
    apb.data = rw.data;
    return apb;
endfunction: reg2bus

virtual function void bus2reg(uvm_sequence_item bus_item,
                             ref uvm_reg_bus_op rw);

    apb_seq_item apb;
    if (!$cast(apb, bus_item)) begin
        `uvm_fatal("NOT_APB_TYPE", "Provided bus_item is not of the
correct type")
        return;
    end
    rw.kind = apb.we ? UVM_WRITE : UVM_READ;
    rw.addr = apb.addr;
    rw.data = apb.data;
    rw.status = UVM_IS_OK;
endfunction: bus2reg

endclass: reg2apb_adapter

```

The adapter code can be found in the file *reg2apb_adapter.svh* in the /agents/apb_agent directory in the example which can be downloaded:

Burst Support

The register model access methods support single accesses to registers and this is in line with the usual use model for register accesses - they are accessed individually, not as part of a burst. If you have registers which need to be tested for burst mode accesses, then the recommended approach is to initiate burst transfers from a sequence running directly on the target bus agent.

If you use burst mode to access registers, then the predictor implementation needs to be able to identify bursts and convert each beat into a predict() call on the appropriate register.

Common Adapter Issues

The adapter is a critical link in the sequence communication chain between the register model access methods and the driver. The expectation is that the driver-sequence API is cleanly implemented and follows either a get_next_item()/item_done() or a get()/put(rsp) completion model on the driver side. The provides_responses bit should be set to 1 if your driver is using put(rsp) to return responses, and 0 if it does not.

If you get the setting of provides_responses wrong, then one of two things will happen - either the stimulus generation will lock up or you will get an immediate return from front door access methods with the wrong responses but see bus activity occur some time later.

The lock up occurs if you are using the get_next_item()/item_done() completion model and you have set the provides_responses bit to a 1 - the reason being that the adapter is waiting for a response that will never be returned.

The instant return occurs if you are using the `get()/put(rsp)` completion model and you have set the `provides_responses` bit to 0 and the `get()` call in the driver code immediately unblocks the sequencer completing the access method. The driver then goes on to execute the bus access before returning the response via the `put(rsp)` method. The adapter ignores the response.

You should also ensure that both read and write accesses follow the same completion model. In particular, make sure that you return a response from the driver for write accesses when using the `get()/put(rsp)` completion model.

Context for the Bus

If the bus protocol targeted by the adapter needs more information to function properly, a method exists for providing context information to the adapter. Please visit the [Adapter Context](#) article for more information.

Integrating a UVM Register Model in a TestBench - Overview

Register Model Testbench Integration - Testbench Architecture Overview

Within an UVM testbench a register model is used either as a means of looking up a mirror of the current DUT hardware state or as means of accessing the hardware via the front or back door and updating the register model database.

For those components or sequences that use the register model, the register model has to be constructed and its handle passed around using a configuration object or a resource. Components and sequences are then able to use the register model handle to call methods to access data stored within it, or to access the DUT.

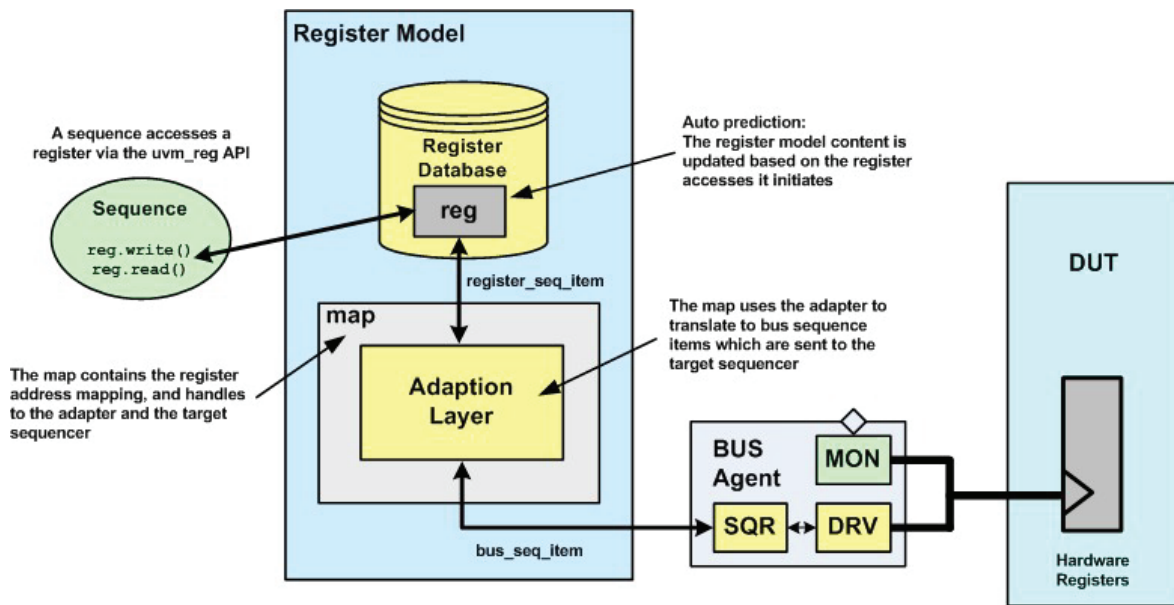
In order to make back door accesses to the DUT, the register model uses hdl paths which are used by simulator runtime database access routines to peek and poke the hardware signals corresponding to the register. The register model is updated automatically at the end of each back door access cycle. The way in which this update is done is by calling the `predict()` method which updates the accessed registers mirrored value. In order for back door accesses to work, no further integration with the rest of the testbench structure is required.

The register model supports front door accesses to the DUT by generating generic register transactions which are converted to target bus agent specific `sequence_items` before being sent to the target bus agents sequencer, and by converting any returned response information back from bus agent `sequence_items` into register transactions. This bidirectional conversion process takes place inside an adapter class which is specific to the target bus agent. There is really only one way in which the stimulus side of the access is integrated with the testbench, but the update, or prediction, of the register model content at the end of a front door access can occur using one of three models and these are:

- Auto Prediction
 - Explicit Prediction
 - Passive Prediction
-

Auto Prediction

Auto prediction is the simplest prediction regime for register accesses. In this mode, the various access methods which cause front door accesses to take place automatically call a `predict()` method using either the data that was written to the register, or the data read back from the register at the end of the bus cycle.



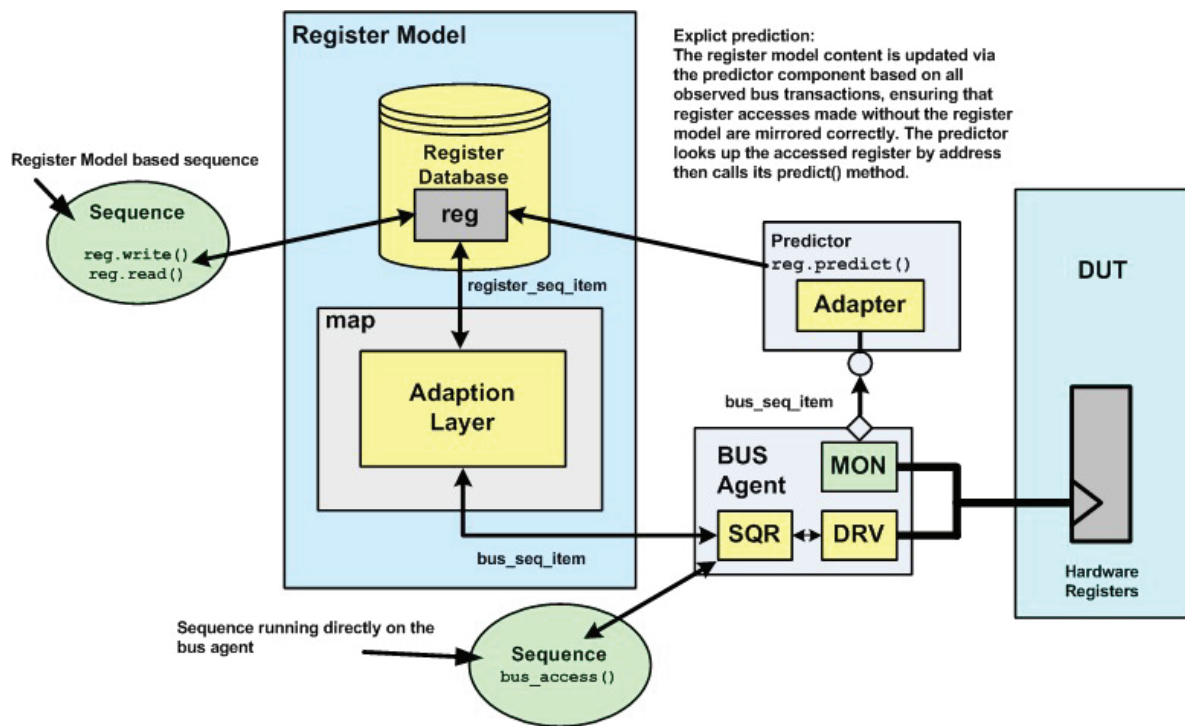
Auto Prediction Use Model

This mode of operation is the simplest to implement, but suffers from the drawback that it can only keep the register model up to date with the transfers that it initiates. If any other sequences directly access the target sequencer to update register content, or if there are register accesses from other DUT interfaces, then the register model will not be updated.

Explicit prediction is the default mode of prediction, to enable auto prediction, use the `set_auto_predict()` method. Also note that when using auto prediction, if the status returned is `UVM_NOT_OK`, the register model will not be updated.

Explicit Prediction (Recommended Approach)

In the explicit prediction mode of operation an external predictor component is used to listen for target bus agent analysis transactions and then to call the `predict()` method of the accessed register to update its mirrored value. The predictor component uses the adapter to convert the bus analysis transaction into a register transaction, then uses the address field to look up the target register before calling its `predict()` method with the data field. Explicit prediction is the default mode of prediction, to disable it use the `set_auto_predict()` method.



Explicit Prediction Use Model

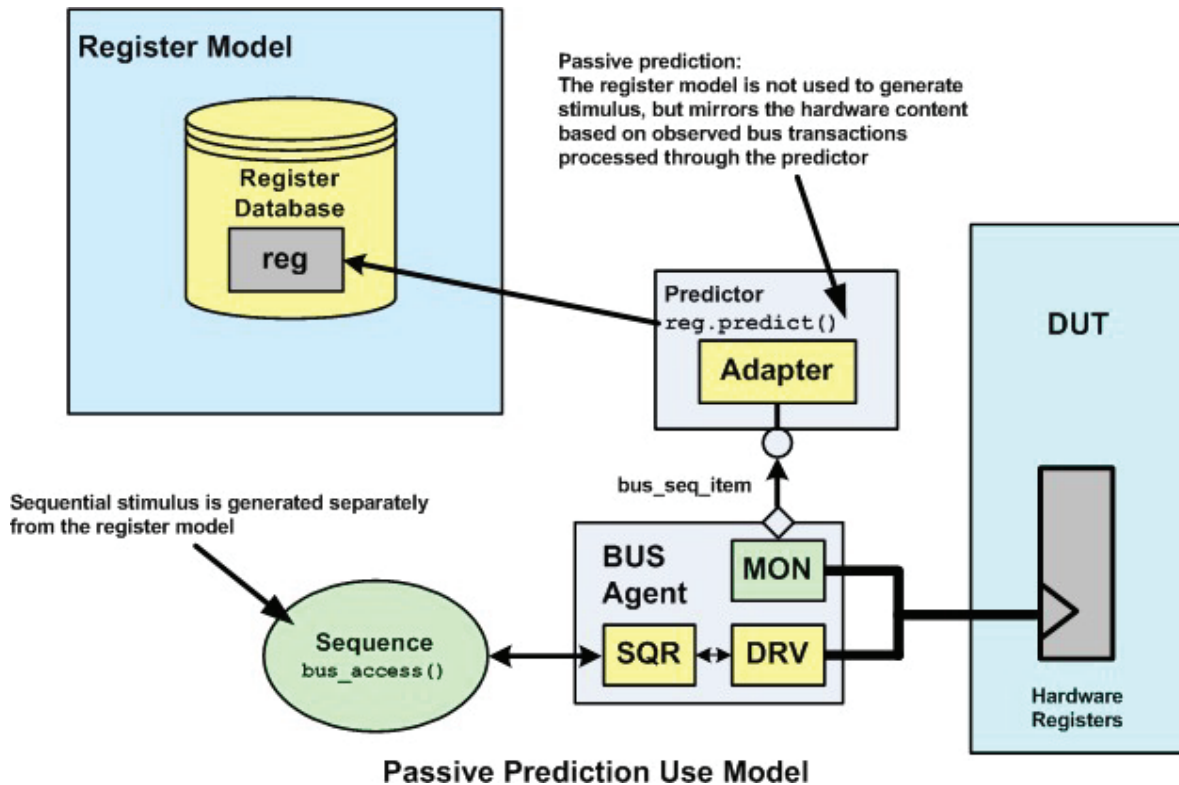
The main advantage of using explicit prediction is that it keeps the register model up to date with all accesses that occur on the target bus interface. The configuration also has more scope for supporting vertical reuse where accesses to the DUT may occur from other bus agents via bus bridges or interconnect fabrics.

During vertical reuse, an environment that supports explicit prediction can also support passive prediction as a result of re-configuration.

Also note that when using explicit prediction, the status value returned to the predictor is ignored. This means that if an errored (UVM_NOT_OK) status is being returned from a register access, the register access will need to be filtered out before sending information to the Predictor if that errored transfer is not meant to update the mirrored value of a register. This could be done in a monitor or with a modified testbench predictor component placed between a monitor and a Predictor.

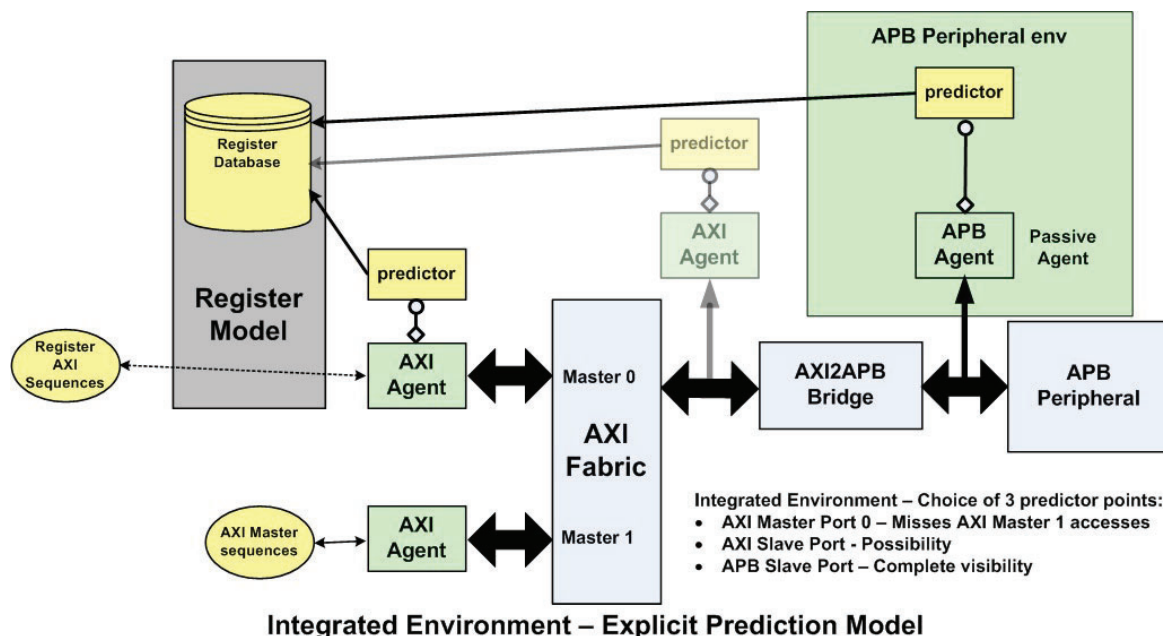
Passive Prediction

In passive prediction the register model takes no active part in any accesses to the DUT, but it is kept up to date by the predictor when any front door register accesses take place.



Vertically Integrated Environment Example

In the integrated environment example in the picture below, the APB peripheral is connected to an AXI interconnect fabric via a bus bridge. The peripheral can be accessed by either of the two bus masters, one of which runs sequences directly and the other uses the register model to generate accesses.



In this scenario there are a number of places where the predictor could be placed. If it is placed on the Master 0 AXI agent, then the register model is updated only as a result of accesses that it generates. If the predictor is placed on the slave AXI port agent, then the predictor will be able to update the register model on accesses from both AXI masters. However, if the predictor is placed on the APB bus it will be able to verify that the correct address mapping is being used for the APB peripheral and that bus transfers are behaving correctly end to end.

In this example, there could also be up to three address translations taking place (AXI Master 0 to AXI Slave, AXI Master 1 to AXI Slave, AXI Slave to APB) and the predictor used would need to use the right register model map when making the call to `predict()` a target registers mirrored value.

Integrating a register model - Implementation Process

The recommended register model integration approach is to use explicit prediction, since this has a number of advantages, not least of which is that it will facilitate vertical reuse.

Based on the assumption that you have a register model, then you will need to follow the steps below in the order described to integrate a register model into your environment:

1. Understand which map in the register model correspond to which target bus agent.
2. Check whether the bus agent has an UVM register model adapter class, otherwise you will have to implement one.
3. Declare and build the register model in the test, passing a handle to it down the testbench hierarchy via configuration (or resources).
4. In each env that contains an active bus interface agent set up the bus layering.
5. In each env that contains a bus agent set up a predictor.

The register model you have will contain one or more maps which define the register address mapping for a specific bus interface. In most cases, block level testbenches will only require one map, but the register model has been designed to cope with the situation where a DUT has multiple bus interfaces which could quite easily occur in a multi-master SoC. Each map will need to have the adapter and predictor layers specified.

Integrating a UVM Register Model in a TestBench - Implementation

The integration process for the register model involves constructing it and placing handles to it inside the relevant configuration objects, and then creating the adaption layers.

Register Model Construction

The register model should be constructed in the test and a handle to it should be passed to the rest of the testbench hierarchy via configuration objects. In the case of a block level environment, a handle to the whole model will be passed. However, in the case of a cluster (sub-system) or a SoC environment, the overall register model will be an integration of register blocks for the sub-components and handles to sub-blocks of the register model will be relevant to different sub-environments.

For instance, in the case of a block level testbench for the SPI, the SPI env configuration object would contain a handle for the SPI register model which would be created and assigned in the test.

The following code is from the test base class for the SPI block level testbench, note that the register model is created using the factory, and then its build() method is called.

Note: The register model build() method will not be called automatically by the uvm_component build() phase. The register model is **NOT** a uvm_component, so its build() method has to be called explicitly to construct and configure the register model. If the register model build method is not called, the objects within the register model are not constructed and initialized which means that subsequent testbench code that accesses the register model will fail, most likely with null handle errors.

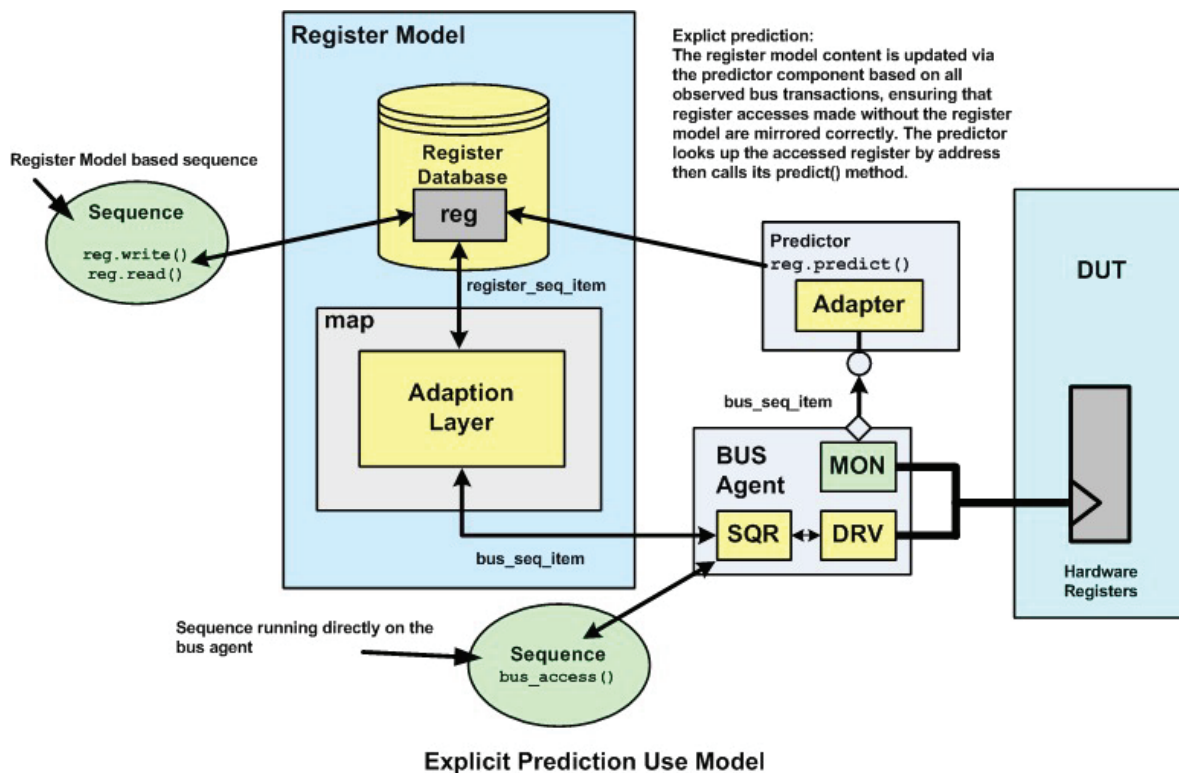
```
//
// From the SPI Test base class
//
// Build the env, create the env configuration
// including any sub configurations and assigning virtual interfaces
function void spi_test_base::build_phase(uvm_phase phase);
    // env configuration
    m_env_cfg = spi_env_config::type_id::create("m_env_cfg");
    // Register model
    // Enable all types of coverage available in the register model
    uvm_reg::include_coverage("*", UVM_CVR_ALL);
    // Create the register model:
    spi_rm = spi_reg_block::type_id::create("spi_rm");
    // Build and configure the register model
    spi_rm.build();
    // Assign a handle to the register model in the env config
    m_env_cfg.spi_rm = spi_rm;
    // etc.
endfunction: build_phase
```

In the case where the SPI is part of a cluster, then the whole cluster register model (containing the SPI register model as a block) would be created in the test, and then the SPI env configuration object would be passed a handle to the SPI register block as a sub-component of the overall register model:


```
//
// From the build method of the PSS test base class:
//
//
// PSS - Peripheral sub-system with a hierarchical register model with
the handle pss_reg
//
// SPI is a sub-block of the PSS which has a register model handle in
its env config object
//
m_spi_env_cfg.spi_rm = pss_reg.spi;
```

Adaption Layer Implementation

There are two parts to the register adaption layer, the first part implements the sequence based stimulus layering and the second part implements the analysis based update of the register model using a predictor component.



Register Sequence Adaption Layer

The register sequence layering adaption should be done during the UVM connect phase when the register model and the target agent components are known to have been built.

The register layering for each target bus interface agent supported by the register model should only be done once for each map. In a block level environment, this will be in the env, but in environments working at a higher level of integration this mapping should be done in the top level environment. In order to determine whether the particular env is at the top level the code should test whether the parent to its register block is null or not - if it is, then the model and therefore its env is at the top level.

In the following code from the SPI block level env connect method, the APB map from the SPI register block is layered onto the APB agent sequencer using the `reg2apb` adapter class using the `set_sequencer()` method in

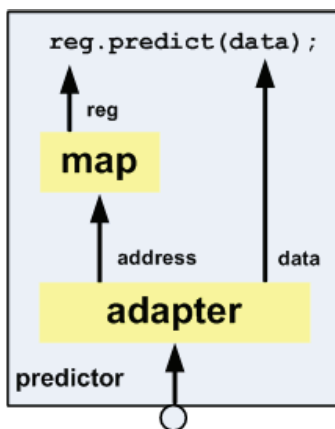
APB_map object:

```
//
// From the SPI env
//
// Register layering adapter:
reg2apb_adapter reg2apb;

function void spi_env::connect_phase(uvm_phase phase);
    if(m_cfg.m_apb_agent_cfg.active == UVM_ACTIVE) begin
        reg2apb = reg2apb_adapter::type_id::create("reg2apb");
        //
        // Register sequencer layering part:
        //
        // Only set up register sequencer layering if the top level env
        if(m_cfg.spi_rm.get_parent() == null) begin
            m_cfg.spi_rm.APB_map.set_sequencer(m_apb_agent.m_sequencer,
            reg2apb);
        end
    end
endfunction: connect_phase
```

Register Prediction

By default, the register model uses a process called explicit_prediction to update the register data base each time a read or write transaction that the model has generated completes. Explicit prediction requires the use of a uvm_reg_predictor component.



The uvm_reg_predictor component is derived from a uvm_subscriber and is parameterized with the type of the target bus analysis transaction. It contains handles for the target register bus adapter and the register model address map that are being used to interface to the bus agent sequencer. It uses the register adapter to convert the analysis transaction from the monitor to a register transaction, then it looks up the register by address in the register model's bus specific map and modifies the contents of the appropriate register.

The uvm_reg_predictor component is part of the UVM library and does not need to be extended. However, to integrate it the following things need to be taken care of:

1. Declare the predictor using the target bus sequence_item as a class specialization parameter
2. Create the predictor in the env build() method
3. In the connect method - set the predictor map to the target register model register map

4. In the connect method - set the predictor adapter to the target agent adapter class
5. In the connect method - connect the predictor analysis export to the target agent analysis port

A predictor should be included at every place where there is a bus monitor on the target bus. The code required is shown below and is from the second half of the SPI connect method code.

```
//
// From the SPI env
//
// Register predictor:
uvm_reg_predictor #(apb_seq_item) apb2reg_predictor;

function void spi_env::build_phase(uvm_phase phase);
    if(!uvm_config_db #(spi_env_config)::get(this, "", "spi_env_config", m_cfg)) begin
        `uvm_error("build_phase", "SPI env configuration object not found")
    end
    if(m_cfg.has_apb_agent) begin
        set_config_object("m_apb_agent*", "apb_agent_config",
m_cfg.m_apb_agent_cfg, 0);
        m_apb_agent = apb_agent::type_id::create("m_apb_agent", this);
        // Build the register model predictor
        apb2reg_predictor = uvm_reg_predictor
#(apb_seq_item)::type_id::create("apb2reg_predictor", this); end
//
// etc
//
endfunction:build_phase

function void spi_env::connect_phase(uvm_phase phase);
    //Adapter Created in code shown above

    // Register prediction part:
    //
    // Replacing implicit register model prediction with explicit
prediction
    // based on APB bus activity observed by the APB agent monitor
    // Set the predictor map:
    apb2reg_predictor.map = m_cfg.spi_rm.APB_map;
    // Set the predictor adapter:
    apb2reg_predictor.adapter = reg2apb;
    // Connect the predictor to the bus agent monitor analysis port
    m_apb_agent.ap.connect(apb2reg_predictor.bus_in);

endfunction : connect_phase
```

The code excerpts shown come from the *spi_tb/tests/spi_test_base.svh* file (register model construction) and the *spi_tb/env/spi_env.svh* file (adapter and predictor) from the example download:

"Quirky" Registers

Introduction

Quirky registers are just like any other register described using the register base class except for one thing. They have special (quirky) behavior that either can't be described using the register base class, or is hard to describe using the register base class. The register base class can be used to describe the behavior of many different registers - for example clear-on-read (RC), write-one-to-set (WIS), write-zero-to-set (W0S). These built-in behaviors are set using attributes. Setting the attribute causes the built-in behavior. Built-in behaviors can be used for a majority of most register descriptions, but most verification environments have a small number of special registers with behavior that can't be described but the built-in attributes. These are quirky registers.

Examples of quirky registers include 'clear on the third read', 'read a register that is actually a collection of bits from 2 other registers'. These are registers that have very special behavior. Quirky registers are outside the register base class functionality and are most easily implemented by extending the base class functions or by adding callbacks.

The register base class library is very powerful and offers many ways to change behavior. The easiest implementations extend the underlying register or field class, and redefine certain virtual functions or tasks, like set(), or get(), or read(). In addition to replacing functions and tasks, callbacks can be added. A callback is called at specific times from within the base class functions. For example, the post_predict() callback is called from the uvm_reg_field::predict() call. Adding a post_predict() callback, allows the field value to be changed (predicted). The IEEE 1800.2 LRM has more information on register function and task overloading as well as callback definition.

Quirky registers built-in to the library

Some special behaviors are delivered as part of the UVM library - uvm_reg_fifo.svh and uvm_reg_indirect.svh. These classes implement fifo behavior and indirect behavior respectively. Building a fifo register can be done by extending this built-in class - uvm_reg_fifo.

```
class fifo_reg extends uvm_reg_fifo;

    function new(string name = "fifo_reg");
        super.new(name, 8, 32, UVM_NO_COVERAGE);
    endfunction: new

    `uvm_object_utils(fifo_reg)

endclass
```

A Custom Quirky Register

If your quirky register behavior does not match either of the library register models, then you'll need to build your own new register functionality.

The example below implements an ID register. An ID register returns an element from a list on each successive read. Each read returns the next item on the list. When the end of the list is reached, the first element is returned. When the ID register is written, the write data causes the list pointer to become the value written. For example, writing 2 to the ID register will cause the third item from the list to be returned on the next read.

ID Register

A snapshot of some code that implements an ID register is below. (See the full example for the complete text).

```

always @(posedge PCLK) begin
  if(PRESETn == 0) begin
    id_register_pointer <= 0;
    id_register_value <= '{ha0, 'ha1, 'ha2, 'ha3, 'ha4,
                          'ha5, 'ha6, 'ha7, 'ha8, 'ha9}; current_value <=
    32'ha0;
  end
  else begin
    if(PSEL & PENABLE) begin
      if(PWRITE) begin
        case(PADDR)
          // A write to the ID register overwrites the register pointer
          `ID : begin
            if(PWDATA < 10) begin
              id_register_pointer <= PWDATA[3:0];
            end
            else begin
              id_register_pointer <= 0;
            end
          end
          `R_W : W_reg <= PWDATA;
        endcase
      end
      else begin
        // A read from the ID register advances the id_register_pointer
        if(PADDR == `ID)begin if(id_register_pointer == 9)
          begin
            id_register_pointer <= 0;
          end
          else begin
            id_register_pointer <= id_register_pointer + 1;
          end
        end
      end
    end
    if(~PENABLE) begin
      current_value <= id_register_value[id_register_pointer];
    end
  end
end

always @(*) begin
  if(PSEL) begin
    case(PADDR)
      // Return the current_value of the id register

```

```

    `ID: PRDATA = current_value;
    `R_W: PRDATA = R_reg;
    default: PRDATA = 0;
endcase
end
else begin
    PRDATA = 0;
end
end
end

```

ID Register Model

The ID register model is implemented below. The register itself is similar to a regular register, except the ID register uses a new kind of field - the `id_register_field`.

The `id_register_field` implements the specific functionality of the ID register in this case.

```

// The ID Register.
// Just a register which has a special field - the
// ID Register field.
class id_register extends uvm_reg;
    id_register_field F1;

    function new(string name = "id_register");
        super.new(name, 8, UVM_NO_COVERAGE);
    endfunction: new

    virtual function void build();
        F1 = id_register_field::type_id::create("F1",,
            get_full_name());
        F1.configure(this, 8, 0, "RW", 0, 8'ha0, 1, 0, 1);
    endfunction: build

    `uvm_object_utils(id_register)

endclass : id_register

```

The `id_register` builds the ID field in the `build()` routine, just like any other register.

ID Register Model Field

The ID register field is a simple implementation of the functionality required. The `post_read()` and `post_write()` callback methods allow the register field functionality to be modeled.

Strictly speaking, the ID field is not modeling the behavior of an id register, it is checking the DUT implementation of the register.

```

// The ID Register (field).
// Each successive 'read' operation returns the next item
// from a list. When the end of the list is reached, it wraps
// around to the beginning.
// This list is a0, a1, ...a9. (10 values)
class id_register_field extends uvm_reg_field;

```

```

`uvm_object_utils(id_register_field)

int id_register_pointer = 0;
int id_register_pointer_max = 10;
int id_register_value[] =
    {'ha0, 'ha1, 'ha2, 'ha3, 'ha4,
     'ha5, 'ha6, 'ha7, 'ha8, 'ha9};

int current_value;

function new(string name = "id_register_field");
    super.new(name);
    current_value = id_register_value[0];
endfunction

task post_read(uvm_reg_item rw);
    if(value != current_value) begin
        `uvm_error("ID_REG_CHECK", $sformatf("Wrong ID value: id_ptr:%0d
id_val:%0h read_vale:%0h", id_register_pointer, current_value, value))
    end
    id_register_pointer++;
    if (id_register_pointer >= id_register_pointer_max) begin
        id_register_pointer = 0;
    end
    current_value = id_register_value[id_register_pointer];
endtask

task post_write(uvm_reg_item rw);
    id_register_pointer = value;
    if (id_register_pointer >= id_register_pointer_max) begin
        id_register_pointer = 0;
    end
    current_value = id_register_value[id_register_pointer];
endtask

endclass

```

When the `post_read()` method is called, it first checks that the `current_value` of the field is the same as the value read back from the DUT. Then it updates the `current_value` of the field, according to the incremented value of the `id_register_pointer`, ready for the next register read.

The `post_write()` method, writes the value into the `id_register_pointer` and sets up the `current_value` for the next read cycle.

The ID register field also contains the specific values that will be read back. Those values could be obtained externally, or could be set from some other location.

Register Model Coverage

Controlling the build/inclusion of covergroups in the register model

Which covergroups get built within a register block object or a register object is determined by a local variable called `m_has_cover`. This variable is of type `uvm_coverage_model_e` and it should be initialized by a `build_coverage()` call within the constructor of the register object which assigns the value of the `include_coverage` resource to `m_has_cover`. Once this variable has been set up, the covergroups within the register model object should be constructed according to which coverage category they fit into.

The construction of the various covergroups would be based on the result of the `has_coverage()` call.

As each covergroup category is built, the `m_cover_on` variable needs to be set to enable coverage sampling on that set of covergroups, this needs to be done by calling the `set_coverage()` method.

Controlling the sampling of covergroups in the register model

Depending on whether the covergroup needs to be sampled automatically on register access or as the result of an external `sample_values()` call, two different methods need to be implemented for each object. The `sample()` method is called automatically for each register and register block on each access. The `sample_values()` method is intended to be called from elsewhere in the testbench. In both cases, these methods test the `m_cover_on` bit field using the `has_coverage()` method to determine whether to sample a set of covergroups or not.

The register model coverage control methods

The various methods used to control covergroup build and their effects are summarized here:

Method	Description
Overall Control	
<code>uvm_reg::include_coverage(uvm_coverage_model_e)</code>	Static method that sets up a resource with the key "include_coverage". Used to control which types of coverage are collected by the register model
Build Control	
<code>build_coverage(uvm_coverage_model_e)</code>	Used to set the local variable <code>m_has_cover</code> to the value stored in the resource database against the "include_coverage" key
<code>has_coverage(uvm_coverage_model_e)</code>	Returns true if the coverage type is enabled in the <code>m_has_cover</code> field
<code>add_coverage(uvm_coverage_model_e)</code>	Allows the coverage type(s) passed in the argument to be added to the <code>m_has_cover</code> field
Sample Control	
<code>set_coverage(uvm_coverage_model_e)</code>	Enables coverage sampling for the coverage type(s), sampling is not enabled by default
<code>get_coverage(uvm_coverage_model_e)</code>	Returns true if the coverage type(s) are enabled for sampling

Note: That it is not possible to set an enable coverage field unless the corresponding build coverage field has been set.

The values of the `uvm_coverage_model_e` enumerated type are:

uvm_coverage_model_e enum value	Description
UVM_NO_COVERAGE	No coverage enabled
UVM_CVR_REG_BITS	Coverage enabled for register bits
UVM_CVR_ADDR_MAP	Coverage enabled for individual register and memory addresses
UVM_CVR_FIELD_VALS	Coverage enabled for register fields
UVM_CVR_ALL	All types of register coverage are enabled

An example

The following code comes from a register model implementation of a register model that incorporates functional coverage.

In the test, the overall register coverage model for the testbench has to be set using the `uvm_reg::include_coverage()` static method:

```
//
// Inside the test build method:
//
function void spi_test_base::build();
    uvm_reg::include_coverage(UVM_CVR_ALL); // All register coverage
types enabled
//
//..
```

The first code excerpt is for a covergroup which is intended to be used at the block level to get read and write access coverage for a register block. The covergroup has been wrapped in a class included in the register model package, this makes it easier to work with.

```
//
// A covergroup (wrapped in a class) that is designed to get the
// register map read/write coverage at the block level
//
//
// This is a register access covergroup within a wrapper class
//
// This will need to be called by the block sample method
//
// One will be needed per map
//
class SPI_APB_reg_access_wrapper extends uvm_object;

    `uvm_object_utils(SPI_APB_reg_access_wrapper)

    covergroup ra_cov(string name) with function sample(uvm_reg_addr_t
addr, bit is_read);

    option.per_instance = 1;
    option.name = name;
```



```

// To be generated:
//
// Generic form for bins is:
//
// bins reg_name = {reg_addr};
ADDR: coverpoint addr {
    bins rxtx0 = {'h0};
    bins rxtx1 = {'h4};
    bins rxtx2 = {'h8};
    bins rxtx3 = {'hc};
    bins ctrl = {'h10};
    bins divider = {'h14};
    bins ss = {'h18};
}

// Standard code - always the same
RW: coverpoint is_read {
    bins RD = {1};
    bins WR = {0};
}

ACCESS: cross ADDR, RW;

endgroup: ra_cov

function new(string name = "SPI_APB_reg_access_wrapper");
    ra_cov = new(name);
endfunction

function void sample(uvm_reg_addr_t offset, bit is_read);
    ra_cov.sample(offset, is_read);
endfunction: sample

endclass: SPI_APB_reg_access_wrapper

```

The second code excerpt is for the register block which includes the covergroup. The code has been stripped down to show the parts relevant to the handling of the coverage model.

In the constructor of the block there is a call to `build_coverage()`, this ANDs the coverage enum argument supplied with the overall testbench coverage setting, which has been set by the `uvm_reg::include_coverage()` method and sets up the `m_has_cover` field in the block with the result.

In the blocks `build()` method, the `has_coverage()` method is used to check whether the coverage model used by the access coverage block is enabled. If it is, the access covergroup is built and then the coverage sampling is enabled for its coverage model using `set_coverage()`.

In the `sample()` method, the `get_coverage()` method is used to check whether the coverage model is enabled for sampling, and then the covergroup is sampled, passing in the address and `is_read` arguments.

```

//
// The relevant parts of the spi_rm register block:
//

//-----
// spi_reg_block

//-----

class spi_reg_block extends uvm_reg_block;
  `uvm_object_utils(spi_reg_block)

  rand rxtx0 rxtx0_reg; rand
  rxtx1 rxtx1_reg; rand rxtx2
  rxtx2_reg; rand rxtx3
  rxtx3_reg; rand ctrl ctrl_reg;
  rand divider divider_reg;
  rand ss ss_reg;

  uvm_reg_map APB_map; // Block map

  // Wrapped APB register access covergroup
  SPI_APB_reg_access_wrapper SPI_APB_access_cg;

//-----

// new

//-----

function new(string name = "spi_reg_block");
  // build_coverage ANDs UVM_CVR_ADDR_MAP with the value set
  // by the include_coverage() call in the test bench
  // The result determines which coverage categories can be
built by this
  // region of the register model
  super.new(name, build_coverage(UVM_CVR_ADDR_MAP));
endfunction

//-----

// build

//-----

virtual function void build();
  string s;

  // Check that the address coverage is enabled

```

```

    if(has_coverage(UVM_CVR_ADDR_MAP)) begin
        SPI_APB_access_cg =
SPI_APB_reg_access_wrapper::type_id::create("SPI_APB_access_cg");
        // Enable sampling on address coverage
        set_coverage(UVM_CVR_ADDR_MAP);
    end

    //
    // Create, build and configure the registers ...
    //

    APB_map = create_map("APB_map", 'h0, 4, UVM_LITTLE_ENDIAN);

    APB_map.add_reg(rtx0_reg, 32'h00000000, "RW");
    APB_map.add_reg(rtx1_reg, 32'h00000004, "RW");
    APB_map.add_reg(rtx2_reg, 32'h00000008, "RW");
    APB_map.add_reg(rtx3_reg, 32'h0000000c, "RW");
    APB_map.add_reg(ctrl_reg, 32'h00000010, "RW");
    APB_map.add_reg(divider_reg, 32'h00000014, "RW");
    APB_map.add_reg(ss_reg, 32'h00000018, "RW"); add_hdl_path("DUT",
"RTL");

    lock_model();
endfunction: build

// Automatically called when a block access occurs:
function void sample(uvm_reg_addr_t offset, bit is_read, uvm_reg_map map);
    // Check whether coverage sampling is enabled for address
accesses:
    if(get_coverage(UVM_CVR_ADDR_MAP)) begin
        // Sample the covergroup if access is for the APB_map
        if(map.get_name() == "APB_map") begin
            SPI_APB_access_cg.sample(offset, is_read);
        end
    end
endfunction: sample

endclass: spi_reg_block
//

```

Backdoor Accesses

The UVM register model facilitates access to hardware registers in the DUT either through front door accesses or back door accesses. A front door access involves using a bus transfer cycle using the target bus agent, consequently it consumes time, taking at least a clock cycle to complete and so it models what will happen to the DUT in real life. A backdoor access uses the simulator database to directly access the register signals within the DUT, with write direction operations forcing the register signals to the specified value and read direction accesses returning the current value of the register signals. A backdoor access takes zero simulation time since it bypasses the normal bus protocol.

Defining The Backdoor HDL Path

To use backdoor accesses with the UVM register model, the user has to specify the hardware, or hdl, path to the signals that a register model represents. To aid reuse and portability, the hdl path is specified in hierarchical sections. Therefore the top level block would specify a path to the top level of the DUT, the sub-system block would have a path from within the DUT to the sub-system, and a register would have a path specified from within the sub-system. The register level hdl path also has to specify which register bit(s) correspond to the target hdl signal.

As an example, in the SPI master testbench, the SPI master is instantiated as "DUT" in the top level testbench, so the hdl path to the register block (which corresponds to the SPI master) is set to "DUT". Then the control register bits within the SPI master RTL is collected together in a vectored reg called "ctrl", so the hdl path to the control register is DUT.ctrl. The hdl path slice for the control register is set to "ctrl" in the build method of the SPI register block.

```
function void spi_reg_block::build();
    //
    // ....
    //
    ctrl_reg = ctrl::type_id::create("ctrl");
    ctrl_reg.build();
    // Can add the hdl path as last argument to configure but only
    if the whole register
        // content is contained within the hdl path
        ctrl_reg.configure(this, null, "");
        // Add the ctrl hdl_path starting at bit 0, hardware target is
14 bits wide
        ctrl_reg.add_hdl_path_slice("ctrl", 0, 14);
        //
        // ....
        //
        // Assign DUT to the hdl path
        add_hdl_path("DUT", "RTL");
        lock_model();
endfunction: build
```

Tradeoffs Between Front And Backdoor Accesses

Backdoor accesses should be used carefully. The following table summarises the key functional differences between the two access modes:

Backdoor Access	Frontdoor Access
Take zero simulation time	Use a bus transaction which will take at least one RTL clock cycle
Write direction accesses force the HW register bits to the specified value	Write direction accesses do a normal HW write
Read direction accesses return the current value of the HW register bits	Read direction accesses to a normal HW read, data is returned using the HW data path
In the UVM register model, backdoor accesses are always auto-predicted - the mirrored value reflects the HW value	Frontdoor accesses are predicted based on what the bus monitor observes
Only the register bits accessed are affected, side effects may occur when time advances depending on HW implementation	Side effects are modelled correctly
Bypasses normal HW	Simulates real timing and event sequences, catches errors due to unknown interactions

Backdoor access can be a useful and powerful technique and some valid use models include:

- Configuration or re-configuration of a DUT - Putting it into a non-reset random state before configuring specific registers via the front door
- Adding an extra level of debug when checking data paths - Using a backdoor peek after a front door write cycle and before a front door read cycle can quickly determine whether the write and read data path is responsible for any errors
- Checking a buffer before it is read by front door accesses - Traps an error earlier, especially useful if the read process does a data transform, or has side-effects

Some invalid use models include:

- Use as an accelerator for register accesses - May be justified if there are other test cases that thoroughly verify the register interface
- Checking the DUT against itself- A potential pitfall whereby the DUT behaviour is taken as correct rather than the specified behaviour

Potential Simulator Optimisation Issues

For the backdoor accesses to work, the simulator database VPI access routines need to be able to find the hdl signals. Most simulators optimise away the detailed structural information needed to do this to improve performance. Therefore to be able to use the backdoor access mechanism you will have to turn off these optimisations, at least for the signals that you would like to access via the backdoor.

For Questa the way to do this is to compile the design with the ***vopt +acc*** switch. See the Questa user documentation for more information.

```
# Compile my design so that all of it has registers visible:
vlog my_design
vopt my_design +acc -o opt
vsim opt +UVM_TESTNAME=reg_backdoor_test
```

Backdoor Access For RTL And Gate Level

The structure of RTL and a gate level netlist is quite different and the same hdl path will not hold in both cases. To address this issue, the register model supports the specification of more than one hdl path. The backdoor hdl_path can be set up for both RTL and Gate level, since the hdl_path methods accept a string argument to group the hdl path definitions into different sets. By default the "RTL" hdl path is used for all hdl_path definitions, but this default can be changed to "GATES" (or anything else) to define hdl_path segments for a gate level netlist. The following code shows how the original example can be extended to add in a "GATES" set of hdl_paths to correspond to a gate level netlist

```
// Adding hdl paths for the gate level of abstraction
function void spi_reg_block::build();
    //
    // ....
    //
    ctrl_reg = ctrl::type_id::create("ctrl");
    ctrl_reg.build();
    // Can add the hdl path as last argument to configure but only
    if the whole register
        // content is contained within the hdl path
        ctrl_reg.configure(this, null, "");
        // Add the ctrl hdl_path starting at bit 0, hardware target is
        14 bits wide
        ctrl_reg.add_hdl_path_slice("ctrl", 0, 14); // "RTL" by
    default
        ctrl_reg.add_hdl_path_slice("ctrl_dff.q", 0, 14, "GATES"); //
    Gate level spec
    //
    // ....
    //
    // Assign DUT to the hdl path for both abstractions
    add_hdl_path("DUT", "RTL");
    add_hdl_path("DUT", "GATES");
    lock_model();
endfunction: build
```

Generating Register Models

A register model can be written by hand, following the pattern given for the SPI master example. However, with more than a few registers this can become a big task and is always a potential source of errors. There are a number of other reasons why using a generator is helpful:

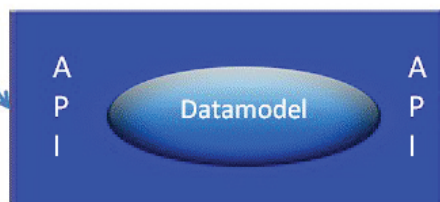
- It allows a common register specification to be used by the hardware, software and verification engineering teams
- The register model can be generated efficiently without errors
- The register model can be re-generated whenever there is a change in the register definition
- Multiple format register definitions for different design blocks can be merged together into an overall register description

There are a number of register generators available commercially, including Mentor's Register Assistant.

As an option to Mentor's Questa Simulator, Register Assistant reads in register descriptions and generates UVM register models and associated register documentation. Register descriptions can be read from spreadsheet (CSV), IP-XACT, and XML format inputs or via API commands within a script. If there are any changes to the register descriptions, the register package and documentation can be automatically updated. Register Assistant has an open datamodel that allows users to add their own readers, writers, or input checks. This capability combined with the API allows every aspect of the tool to be customized.

Register Description

A	B	C	D	E	F	G	H	I
1	Register ID	Register Name	Register Description	Register O	Register A	Register V	Register R	Register Is
2	stopwatch_value	Current value	0x0	32	read-only	TRUE	0x0	
3	stopwatch_reset_value	Reset value	0x04	32	read-write	TRUE	0x0	
4	stopwatch_upper_limit	Upper limit	0x08	32	read-write	TRUE	0x0	
5	stopwatch_lower_limit	Lower limit	0x0C	32	read-write	TRUE	0x0	
6	stopwatch_memory	Memory register	0x010	32	read-write	TRUE	0x0	
7	stopwatch_csr	Control Status Register	0x030	32	read-write	TRUE	0xEFFFFFFF	
8	stopwatch_csr	Control Status Register	0x030	32	read-write	TRUE	0xEFFFFFFF	
9	stopwatch_csr	Control Status Register	0x030	32	read-write	TRUE	0xEFFFFFFF	
10	stopwatch_csr	Control Status Register	0x030	32	read-write	TRUE	0xEFFFFFFF	
11	stopwatch_csr	Control Status Register	0x030	32	read-write	TRUE	0xEFFFFFFF	
12	stopwatch_csr	Control Status Register	0x030	32	read-write	TRUE	0xEFFFFFFF	



```

import uvm_pkg::*;
`include "uvm_macros.svh"

/* DEFINE REGISTER CLASSES */

//-----
// CHN_SWPTR
//-----
class CHN_SWPTR extends uvm_reg;
    `uvm_object_utils(chn_swptr)

    uvm_reg_field swptr_en;
    uvm_reg_field swptr;
    uvm_reg_field reserved;

    //-----
    // get
    //-----
    function new(string name = "CHN_SWPTR");
        super.new(name, 32, UVM_NO_COVERAGE);
    endfunction

    //-----
    // build
    //-----
    virtual function void build();
        swptr_en = uvm_reg_field::type_id::create("SWPTR_EN");
        swptr = uvm_reg_field::type_id::create("SWPTR");
        reserved = uvm_reg_field::type_id::create("RESERVED");

        swptr_en.configure(this, 1, 31, "RW", 1'b0, "UVM_REG_NEW_DATA_WIDTH'hx, 0, 1);
        swptr.configure(this, 29, 2, "RW", 29'b000000000000000000000000000000, "UVM_REG_NEW_DATA_WIDTH'hx, 0, 1);
        reserved.configure(this, 2, 0, "RO", 2'b00, "UVM_REG_NEW_DATA_WIDTH'hx, 0, 1);
    endfunction
endclass
  
```

Documentation

Block : sw_top_block

Description :

Top block for the stopwatch design

Maps:

Map Name	Is Default	Description
SW_MAP2	yes	SW top block map

Sub Blocks:

Block	Instance Name	Offset	Description
sw_top_block	sw1	0x1000	Block1 instance 1
sw_top_block	sw2	0x2000	Block1 instance 2

Register Instances:

Register	Instance Name	Offset	Permissions	Reset	Description
stopwatch_value	swreg1	0x2000	1	1	Create instance 1
stopwatch_upper_limit	swreg2	0x2004	1	1	Create instance 2
stopwatch_lower_limit	swreg3	0x2008	1	1	Create instance 3
stopwatch_memory	swreg4	0x200C	1	1	Create instance 4

Memory Instances:

Memory	Instance Name	Offset	Range	Width	Reset	Description
stopwatch_memory	mem1	0x20	1024	32	1	Memory instance

Map : SW_MAP2 of Block : sw_top_block

Description :

SW top block map

Address Map:

Physical Address	Bitstream Name	Width (bits)	Direction
0x00001000	sw_top_block.sw1	32	1
0x00001004	sw_top_block.sw1.stopwatch_value	32	1
0x00001008	sw_top_block.sw1.stopwatch_upper_limit	32	1
0x0000100C	sw_top_block.sw1.stopwatch_lower_limit	32	1
0x00001010	sw_top_block.sw1.stopwatch_memory	32	1
0x00001014	sw_top_block.sw1.stopwatch_csr	32	1
0x00001018	sw_top_block.sw1.stopwatch_csr	32	1
0x0000101C	sw_top_block.sw1.stopwatch_csr	32	1
0x00001020	sw_top_block.sw1.stopwatch_csr	32	1
0x00001024	sw_top_block.sw1.stopwatch_csr	32	1
0x00001028	sw_top_block.sw1.stopwatch_csr	32	1
0x0000102C	sw_top_block.sw1.stopwatch_csr	32	1
0x00001030	sw_top_block.sw1.stopwatch_csr	32	1
0x00001034	sw_top_block.sw1.stopwatch_csr	32	1
0x00001038	sw_top_block.sw1.stopwatch_csr	32	1
0x0000103C	sw_top_block.sw1.stopwatch_csr	32	1

Register Assistant Functionality

Register-Level Stimulus

Stimulus Abstraction

Stimulus that accesses memory mapped registers should be made as abstract as possible. The reasons for this are that it:

- Makes it easier for the implementer to write
- Makes it easier for users to understand
- Provides protection against changes in the register map during the development cycle
- Makes the stimulus easier to reuse

Of course, it is possible to write stimulus that does register reads and writes directly via bus agent sequence items with hard coded addresses and values - for instance `read(32'h1000_f104);` or `write(32'h1000_f108, 32'h05);` - but this stimulus would have to be re-written if the base address of the DUT changed and has to be decoded using the register specification during code maintenance.

The register model contains the information that links the register names to their addresses, and the register fields to their bit positions within the register. This means the register model makes it easier to write stimulus that is at a more meaningful level of abstraction - for instance `read(SPI.ctrl);` or `write(SPI.rxtx0, 32'h0000_5467);`.

The register model allows users to access registers and fields by name. For instance, if you have a SPI register model with the handle `spi_rm`, and you want to access the control register, `ctrl`, then the path to it is `spi_rm.ctrl`. If you want to access the `go_bsy` field within the control register then the path to it is `spi_rm.ctrl.go_bsy`.

Since the register model is portable, and can be quickly regenerated if there is a change in the specification of the register map, using the model allows the stimulus code to require minimal maintenance, once it is working.

The register model is integrated with the bus agents in the UVM testbench. What this means to the stimulus writer is that he uses register model methods to initiate transfers to/from the registers over the bus interface rather than using sequences which generate target bus agent sequence items. For the stimulus writer this reduces the amount of learning that is needed in order to become productive.

UVM Register Data Value Tracking

The register model has its own database which is intended to represent the state of the hardware registers. For each register there is a mirrored value and a desired value. The desired value represents a state that the register model is going to use to update the hardware, but has not done so. In other words, the desired value allows the user to setup individual register fields before doing a write transfer. The mirrored value represents the current known state of the hardware register. The mirrored value is updated at the end of front bus read and write cycles either based on the data value seen by the register model (auto-prediction) or based on bus traffic observed by a monitor and sent to predictor that updates the register model content (recommended approach for integrating the register model). Backdoor accesses update the register model automatically. The mirrored value can become out of date over time if any of the bits within it are volatile, in other words, they are changed by hardware events rather than by being programmed.

UVM Register Access Methods

The register model has a number of methods which can be used to read and write registers in the DUT. These methods use the desired and mirrored values to keep in step with the actual hardware register contents.

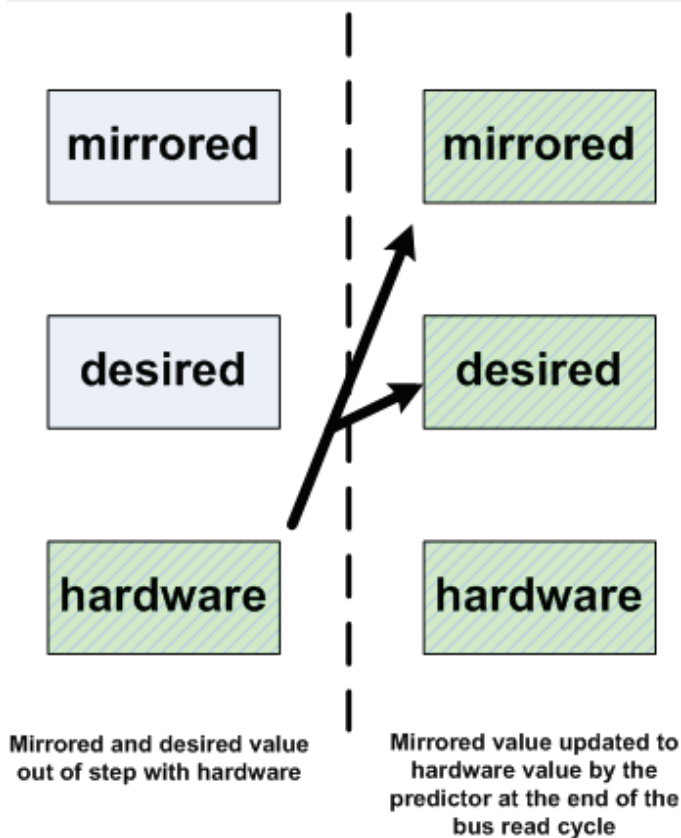
The register model can either use front door or back door access to hardware registers. Front door access involves using the bus interface via an agent and simulates real life bus transfers with their associated timing and event relationships. Back door access uses simulator data base access routines to return the current value of the hardware register bits, or to force them to a particular value. Back door accesses happen in zero simulation time.

read and write

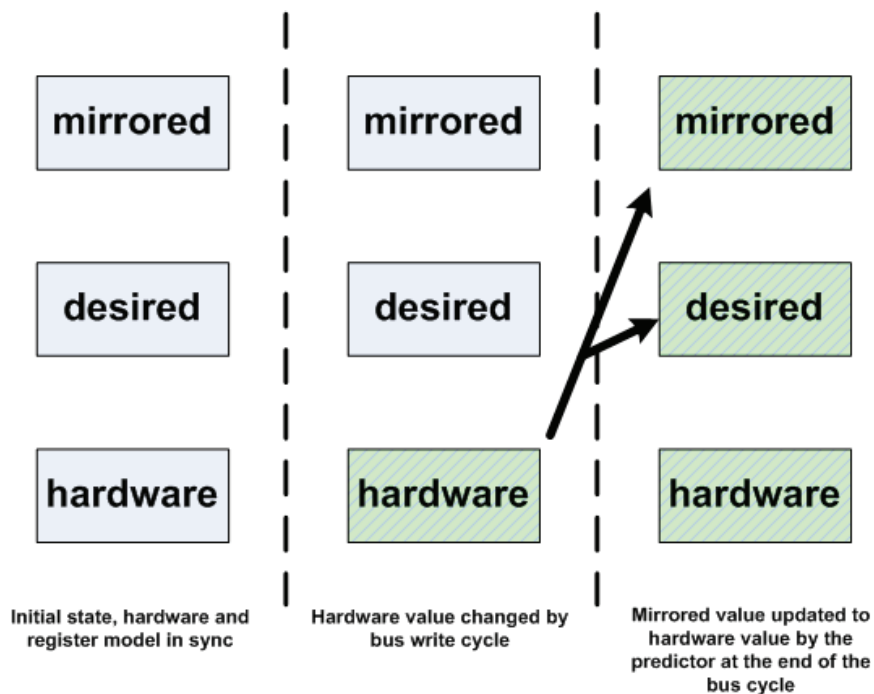
The read() method returns the value of the hardware register. When using front door accesses, calling the read() method results in a bus transfer and the desired and mirrored values of the register model are updated by the bus predictor on completion of the read cycle.

```
//
// read task prototype
//
task read(output uvm_status_e      status,
          output uvm_reg_data_t    value,
          input  uvm_door_e        path = UVM_DEFAULT_DOOR,
          input  uvm_reg_map       map = null,
          input  uvm_sequence_base parent = null,
          input  int               prior = -1,
          input  uvm_object        extension = null,
          input  string            fname = "",
          input  int               lineno = 0);

//
// Example - from within a sequence
//
// Note use of positional and named arguments
//
spi_rm.ctrl.read(status, read data, .parent(this));
```



The write() method writes a specified value to the target hardware register. For front door accesses the mirrored and desired values are updated by the bus predictor on completion of the write cycle.



```
//
// write task prototype
//
task write(output uvm status e      status,
           input uvm_reg_data_t     value,
           input uvm_door_e         path = UVM_DEFAULT_DOOR,
           input uvm reg map        map = null,
           input uvm sequence base parent = null,
           input int                prior = -1,
           input uvm_object         extension = null,
           input string             fname = "",
           input int                lineno = 0);

//
// Example - from within a sequence
//
// Note use of positional and named arguments
//
spi rm.ctrl.write(status, write data, .parent(this));
```

Although the read and write methods can be used at the register and field level, they should only be used at the register level to get predictable and reusable results. Field level reads and writes can only work if the field takes up and fills the whole of a byte lane when the target bus supports byte level access. Whereas this might work with register stimulus written with one bus protocol in mind, if the hardware block is integrated into a sub-system which uses a bus protocol that does not support byte enables, then the stimulus may no longer work.

The read() and write() access methods can also be used for back door accesses, and these complete and update the mirror value immediately.

set and get

The `set()` and `get()` methods operate on the register model and do not result in any accesses to the hardware.

A call to `set` will assign an internal "desired value" for a register or field in the register model. This internal value will be a function of the value argument supplied to `set`, the current mirrored value, and the access policy. Calling `get` after a `set` will return the calculated desired value, which may or may not be the value you provided in the call to `set`.

Some examples:

- For RW access policies, the calculated desired value is always the value argument provided to `set`—the current mirrored value is irrelevant.
- For W1C, the desired value is the bitwise AND of the inverse of the value argument and the current mirror value.
- For RO, the desired value is always the current mirror value, i.e. the 'set' has no effect whatsoever.
- For WC, the desired value is always 0, regardless of the value argument or current mirrored value.

If a `set` does not result in a desired value that is different than the current mirrored value, that field will not require an update. If all fields in a given register do not require an update, the register as a whole will not require an update, and therefore no bus activity will occur for that register upon a call to `update`.

It's worth noting that multiple 'sets' to a register or field are cumulative until the `update` call. Normally, a register or field is set once before a call to `update`. But in cases where there are multiple sets, the register model uses the access policies to update the actual desired value each time. When `update` is called, the final desired value is applied on the bus, if necessary.

For example, for a 16-bit W1T (write 1 to toggle) register:

```
my_reg.set( 16'hFFFF ); // desired is now ~mirrored
my_reg.set( 16'hFFFF ); // desired is now what mirrored was originally
my_reg.update(...);
```

This would produce no bus activity because the calculated desired value is the the mirror value toggled twice, i.e. the cumulative result is the same as the current mirrored value, and so no update to the DUT is needed.

The following table summarizes the behavior of a register set/get operation based on each field's access mode, set value, and current mirror value.

- For register-level set/update operations, the table is applied to each field's `access_mode` independently.
- A bus operation is necessary if any of the field's calculated desired values are different than their current mirror value.
- If multiple sets occur before an update, the mirror value column is the previous calculated desired value, D

UVM Register set/update behavior vs access mode

Access Mode	Set value (A)	Mirror value (B)	Desired value (D)	Bus Access?	Value seen on Bus during update()
RO, RC, RS	x	x	no chg	N	-
RW, WRC, WRS, WO	A	B	A	If (B!=A)	D
WC, WCRS, WOC	x	B	0	If (B!=0)	D
WS, WSRC, WOS	x	B	all 1s	if (B!=1s)	D
W1C, W1CRS	A	B	$\sim A \& B$	If (B != D)	$\sim D$
W1S, W1SRC	A	B	$A B$	If (B != D)	D
W1T	A	B	$A \wedge B$	If (B != D)	$D \wedge B$
W0C, W0CRS	A	B	$A \& B$	If (B != D)	D

W0S, W0SRC	A	B	$\sim A \mid B$	If (B != D)	$\sim D$
W0T	A	B	$\sim A \wedge B$	If (B != D)	$\sim (D \wedge B)$
W1, W01	A	B	first ? A : B Note	If (B != D)	D

Because sets are cumulative until an *update* operation, the Mirror Value column can represent the previously calculated desired value from the last *set* operation. For W1 and W01 modes, the variable ‘first’ in the Desired Value column has a value of one(1'b1) until the first predicted write after a hard reset.

Examples:

The `get()` method returns the calculated desired value of a register or a field.

```
//
// get function prototype
//
function uvm_reg_data_t get(string fname = "",
                           int   lineno = 0);

//
// Examples - from within a sequence
//
uvm_reg_data_t ctrl_value;
uvm_reg_data_t char_len_value;

// Register level get:
ctrl_value = spi_rm.ctrl.get();

// Field level get (char_len is a field within the ctrl reg):
char_len_value = spi_rm.ctrl.char_len.get();
```

The `set()` method is used to setup the desired value of a register or a field prior to a write to the hardware using the `update()` method.

```
//
// set function prototype
//
function void set(uvm_reg_data_t value,
                 string         fname = "",
                 int            lineno = 0);

//
// Examples - from within a sequence
//
uvm_reg_data_t ctrl_value;
uvm_reg_data_t char_len_value;

// Register level set:
spi_rm.ctrl.set(ctrl_value);

// Field level set (char_len is a field within the ctrl reg):
```

```
spi_rm.ctrl.char_len.set(ctrl_value);
```

get_mirrored_value

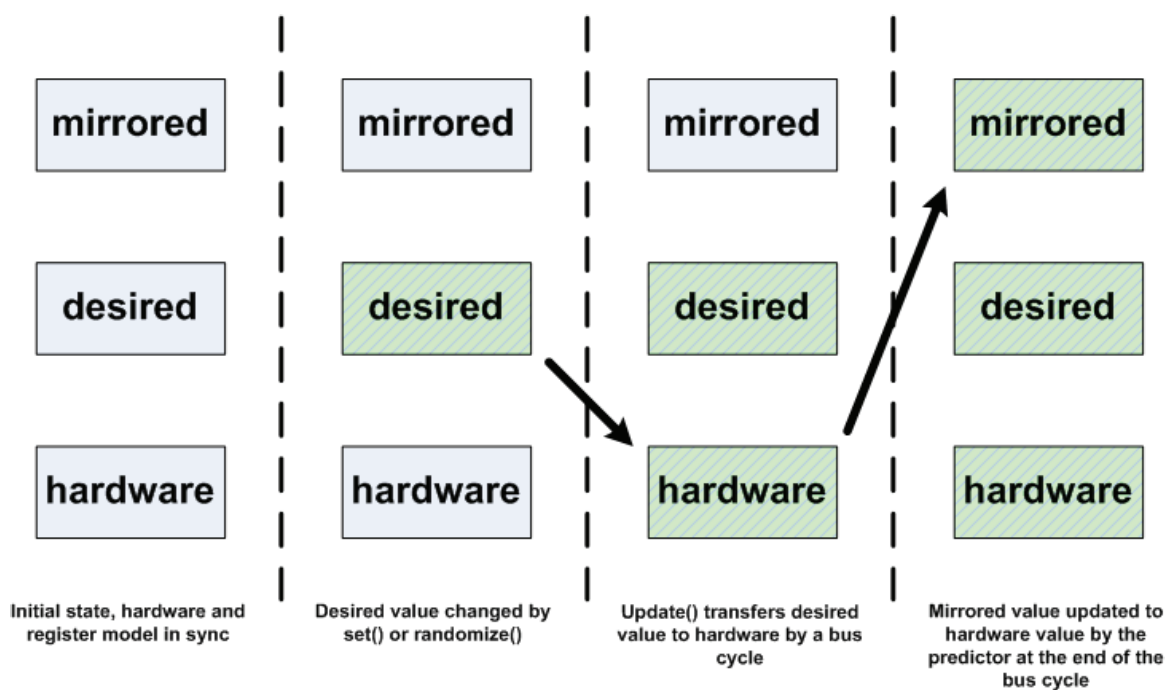
The `get_mirrored_value()` method is used to access the current mirrored value that is stored in the register model, like the `set` and `get` access methods it is an internal access method and does not cause any front or backdoor. It is intended to be used in scenarios where the current mirrored value of the register or field state is required without incurring the overhead of a register access.

update

If there is a difference in value between the desired and the mirrored register values, the `update()` method will initiate a write to a register. The `update()` can be called at the register level which will result in a single register write, or it can be called at the block level in which case it could result in several register writes. The mirrored value for the register would be set to the updated value by the predictor at the completion of the bus write cycle.

```
//
// Prototype for the update task
//
task update(output uvm_status e      status,
            input  uvm_door_e        path = UVM_DEFAULT_DOOR,
            input  uvm_sequence_base parent = null,
            input  int               prior = -1,
            input  uvm_object        extension = null,
            input  string            fname = "",
            input  int               lineno = 0);

//
// Examples:
//
// Block level:
spi_rm.update(status);
//
```



```
// Register level:
spi_rm.ctrl.update(status);
```

Block level updates will always follow the same register access order. The update process indexes the register array in its database and the order used is dependent on the order that registers were created and added to the array by the register model. If randomized or variations in register access ordering are required then you should use individual register updates with your own ordering, perhaps using an array of register handles.

If multiple stimulus streams are active and using update() at the block level, then there is a chance that registers will be updated more than once, since multiple block level update() calls would be made.

peek and poke

The peek() and poke() methods are backdoor access methods which can be used at the field and register level. The peek() method does a direct read of the hardware signal state and the poke() method forces the hardware signal state to match the data value. In both cases, the desired and mirrored values in the register model are updated automatically.

```
//
// peek task prototype
//
task peek(output uvm_status_e      status,
          output uvm_reg_data_t    value,
          input  string            kind = "",
          input  uvm_sequence_base parent = null,
          input  uvm_object        extension = null,
          input  string            fname = "",
          input  int               lineno = 0);

//
// poke task prototype
//
task poke(output uvm_status_e      status,
          input  uvm_reg_data_t    value,
          input  string            kind = "",
          input  uvm_sequence_base parent = null,
          input  uvm_object        extension = null,
          input  string            fname = "",
          input  int               lineno = 0);

//
// Examples - from within a sequence
//
uvm_reg_data_t ctrl_value;
uvm_reg_data_t char_len_value;

// Register level peek:
ctrl_value = spi_rm.ctrl.peek(status, ctrl_value, .parent(this));

// Field level peek (char_len is a field within the ctrl reg):
spi_rm.ctrl.char_len.peek(status, char_len_value, .parent(this));
```

```
// Register level poke:
spi_rm.ctrl.poke(status, ctrl_value, .parent(this));

// Field level poke:
spi_rm.ctrl.char_len.poke(status, char_len_value, .parent(this));
```

randomize

Strictly speaking, this `randomize()` is not a register model method since the register model is based on SystemVerilog class objects. Depending on whether the registers and the register fields have been defined as `rand` or not, they can be randomized with or without constraints. The register model uses the `post_randomize()` method to modify the desired register or field value. Subsequently, the hardware register can be written with the result of the randomization using the `update()` method.

The `randomize()` method can be called at the register model, block, register or field level.

mirror

The `mirror()` method initiates a hardware read or peek access but does not return the hardware data value. A frontdoor read bus level operation results in the predictor updating the mirrored value, whereas a backdoor peek automatically updates the mirrored value. There is an option to check the value read back from the hardware against the original mirrored value.

The `mirror()` method can be called at the field, register or block level. In practice, it should only be used at the register or block level for front door accesses since field level read access may not fit the characteristics of the target bus protocol. A block level `mirror()` call will result in read/peek accesses to all of the registers within the block and any sub-blocks.

```
//
// mirror task prototype:
//
task mirror(output uvm status e      status,
            input  uvm_check_e      check = UVM_NO_CHECK,
            input  uvm_door_e       path  = UVM_DEFAULT_DOOR,
            input  uvm_sequence_base parent = null,
            input  int              prior = -1,
            input  uvm_object       extension = null,
            input  string           fname = "",
            input  int              lineno = 0);

//
// Examples:
//
spi_rm.ctrl.mirror(status, UVM_CHECK); // Check the contents of the
ctrl register
//
spi_rm.mirror(status, .path(UVM_BACKDOOR); // Mirror the contents of
spi_rm block via the backdoor
```

reset

The `reset()` method sets the register desired and mirrored values to the pre-defined register reset value. This method should be called when a hardware reset is observed to align the register model with the hardware. The `reset()` method is an internal register model method and does not cause any bus cycles to take place.

The `reset()` method can be called at the block, register or field level.

```
//
// reset function prototype:
//
function void reset(string kind = "HARD");
//
// Examples:
//
spi_rm.reset(); // Block level reset
//
spi_rm.ctrl.reset(); // Register level reset
//
spi_rm.ctrl.char_len.reset(); // Field level reset
```

get_reset

The `get_reset()` method returns the pre-defined reset value for the register or the field. It is normally used in conjunction with a `read()` or a `mirror()` to check that a register has been reset correctly.

```
//
// get_reset function prototype:
//
function uvm_reg_data_t get_reset(string kind = "HARD");
//
// Examples:
//
uvm_reg_data_t ctrl_value;
uvm_reg_data_t char_len_value;

ctrl_value = spi_rm.ctrl.get_reset(); // Register level
char_len_value = spi_rm.ctrl.char_len.get_reset(); // Field level
```

UVM Register Access Method Arguments

Some of the register access methods contain a large number of arguments. Most of these have a default value so the user does not have to specify all of them when calling a method. The arguments required are more or less the same for those calls that require them and the following table summarizes their purpose:

Argument	Type	Default Value	Purpose
status	uvm_status_e	None, must be populated with an argument	To return the status of the method call - can be UVM_IS_OK, UVM_NOT_OK, UVM_IS_X
value	uvm_reg_data_t	None	To pass a data value, an output in the read direction, an input in the write direction
path	uvm_door_e	UVM_DEFAULT_DOOR	To specify whether a front or back door access is to be used - can be UVM_FRONTDOOR, UVM_BACKDOOR, UVM_PREDICT, UVM_DEFAULT_DOOR
parent	uvm_sequence_base	null	To specify which sequence should be used when communicating with a sequencer
map	uvm_reg_map	null	To specify which register model map to use to make the access
prior	int	-1	To specify the priority of the sequence item on the target sequencer
extension	uvm_object	null	Allows an object to be passed in order to extend the call
fname	string	""	Used by reporting to tie method call to a file name
lineno	int	0	Used by reporting to tie method call to a line number
kind	string	"HARD"	Used to denote the type of reset

UVM Register Access Method Summary

The following table summaries the various register access methods and the level at which they can be used for back door and front door accesses.

	Front door access			Back door access			
Method	Block	Register	Field	Block	Register	Field	Comment
read()	No	Yes	Not recommended	No	Yes	Yes	
write()	No	Yes	Not recommended	No	Yes	Yes	
get()	No	Yes	Yes	No	Yes	Yes	Internal Method
set()	No	Yes	Yes	No	Yes	Yes	Internal Method
peek()	No	Yes	Yes	No	Yes	Yes	
poke()	No	Yes	Yes	No	Yes	Yes	
randomize()	Yes	Yes	Yes	Yes	Yes	Yes	SV Class randomize()
update()	Yes	Yes	No	Yes	Yes	No	
mirror()	Yes	Yes	No	Yes	Yes	No	
reset()	Yes	Yes	Yes	Yes	Yes	Yes	Internal Method
get_reset()	No	Yes	Yes	No	Yes	Yes	Internal Method

UVM Register Access Method Examples

To see examples of how to use the various register model access methods from sequences, please go to the SPI register based sequences example page.

Memory-Level Stimulus

Memory Model Overview

The UVM register model also supports memory access. Memory regions within a DUT are represented by memory models which have a configured width and range and are placed at an offset defined in a register map. The memory model is defined as having either a read-write (RW), a read-only (RO - ROM), or a write-only (WO) access type.

Unlike the register model, the memory model does not store state, it simply provides an access layer to the memory. The reasoning behind this is that storing the memory content would mean incurring a severe overhead in simulation and that the DUT hardware memory regions are already implemented using models which offer alternative verification capabilities. The memory model supports front door accesses through a bus agent, or backdoor accesses with direct access to the memory model content.

Memory Model Access Methods

The memory model supports 4 types of access methods:

- read
- write
- burst_read
- burst_write

Memory read

The read() method is used to read from a memory location, the address of the location is the offset within the memory region, rather than the absolute memory address. This allows stimulus accessing memory to be relocatable, and therefore reusable.

```
//
// memory read method prototype
//
task uvm_mem::read(output uvm_status_e      status,           //
Outcome of the write cycle
                  input  uvm_reg_addr_t     offset,           // Offset
address within the memory region
                  output uvm_reg_data_t     value,           // Read
data
                  input  uvm_door_e         path = UVM_DEFAULT_DOOR, //
Front or backdoor access
                  input  uvm_reg_map        map = null,       // Which
map, memory might in be >1 map
                  input  uvm sequence base parent = null,     // Parent
sequence
                  input  int               prior = -1,        //
Priority on the target sequencer
                  input  uvm_object         extension = null, // Object
allowing method extension
                  input  string             fname = "",        //
Filename for messaging
                  input  int               lineno = 0);        // File
```

```

line number for messaging
//
// Examples:
//
mem_ss.mem_1.read(status, 32'h1000, read_data, .parent(this)); // Using
default map
//
mem_ss.mem_1.read(status, 32'h2000, read_data, .parent(this),
.map(AHB_2_map)); // Using alternative map

```

Memory write

The write() method is used to write to a memory location, and like the read method, the address of the location to be written to is an offset within the memory region.

```

//
// memory write method prototype
//
task uvm_mem::write(output uvm_status_e          status,          //
Outcome of the write cycle
                      input    uvm_reg_addr_t      offset,          //
Offset address within the memory region
                      input    uvm_reg_data_t      value,          // Write
data
                      input    uvm_door_e          path = UVM_DEFAULT_DOOR,
// Front or backdoor access
                      input    uvm_reg_map          map = null,          // Which
map, memory might be in >1 map
                      input    uvm_sequence_base parent = null,          //
Parent sequence
                      input    int                  prior = -1,          //
Priority on the target sequencer
                      input    uvm_object          extension = null, //
Object allowing method extension
                      input    string              fname = "",          //
Filename for messaging
                      input    int                  lineno = 0);          // File
line number for messaging
//
// Examples:
//
mem_ss.mem_1.write(status, 32'h1000, write_data, .parent(this)); //
Using default map
//
mem_ss.mem_1.write(status, 32'h2000, write_data, .parent(this),
.map(AHB_2_map)); // Using alternative map

```

Memory burst_read

The `burst_read()` method is used to read an array of data words from a series of consecutive address locations starting from the specified offset within the memory region. The number of read accesses in the burst is determined by the size of the read data array argument passed to the `burst_read()` method.

```
//
// memory burst_read method prototype
//
task uvm_mem::burst_read(output uvm_status_e      status,           //
Outcome of the write cycle
                        input    uvm_reg_addr_t    offset,           //
Offset address within the memory region
                        output uvm_reg_data_t    value[],           //
Read data array
                        input    uvm_door_e        path =
UVM_DEFAULT_DOOR, // Front or backdoor access
                        input    uvm_reg_map        map = null,      //
Which map, memory might be in >1 map
                        input    uvm_sequence_base parent = null,    //
Parent sequence
                        input    int                prior = -1,       //
Priority on the target sequencer
                        input    uvm_object        extension = null, //
Object allowing method extension
                        input    string            fname = "",        //
Filename for messaging
                        input    int                lineno = 0);      //
File line number for messaging
//
// Examples:
//
uvm_reg_data_t read_data[];
//
// 8 Word transfer:
//
read_data = new[8]; // Set read_data array to size 8
mem_ss.mem_1.burst_read(status, 32'h1000, read_data, .parent(this)); //
    Using default map
//
// 4 Word transfer from an alternative map:
//
read_data = new[4]; // Set read_data array to size 4
mem_ss.mem_1.burst_read(status, 32'h2000, read_data, .parent(this),
.map(AHB_2_map));
```

Memory burst_write

The memory burst write() method is used to write an array of data words to a series of consecutive address locations starting from the specified offset with the memory region. The size of the data array determines the length of the burst.

```
//
// memory burst_write method prototype
//
task uvm_mem::burst_write(output uvm_status_e          status,          //
    Outcome of the write cycle
                                input    uvm_reg_addr_t    offset,      //
    Offset address within the memory region
                                input    uvm_reg_data_t    value[],      //
    Write data array
                                input    uvm_door_e        path =
    UVM_DEFAULT_DOOR, // Front or backdoor access
                                input    uvm_reg_map        map = null,    //
    Which map, memory might be in >1 map
                                input    uvm_sequence_base parent = null,  //
    Parent sequence
                                input    int                prior = -1,      //
    Priority on the target sequencer
                                input    uvm_object        extension = null, //
    Object allowing method extension
                                input    string            fname = "",      //
    Filename for messaging
                                input    int                lineno = 0);    //
    File line number for messaging
//
// Examples:
//
uvm_reg_data_t write_data[];
//
// 8 Word transfer:
//
write_data = new[8]; // Set write_data array to size 8
foreach(write_data[i]) begin
    write_data[i] = i*16;
end
mem_ss.mem_1.burst_write(status, 32'h1000, write_data, .parent(this));
// Using default map
//
// 4 Word transfer from an alternative map:
//
write_data = new[4]; // Set read_data array to size 4 write_data = '{32'h55AA_55AA,
32'hAA55_AA55, 32'h55AA_55AA, 32'hAA55_AA55}';
mem_ss.mem_1.burst_write(status, 32'h2000, write_data, .parent(this),
```

```
.map (AHB_2_map) );
```

Memory Burst Considerations

Exactly how the burst will be implemented is determined by the adapter class that takes care of mapping generic register transactions to/from target agent sequence_items. If stimulus reuse with different target buses is important, care should be taken to make the burst size within a sensible range (e.g. 4, 8 or 16 beats) and to ensure that the start address is aligned to a word boundary. If there is a need to verify that a memory interface can support all potential types of burst and protocol transfer supported by a bus, then this should be done by running native sequences directly on the bus agent rather than using the memory model.

Note: Memory burst accesses are actually broken down into single transfers by the underlying implementation of the register model adaption layer. If you are interested in having to support true burst accesses then you could implement your own extended version of the `uvm_mem` class, overloading the `do_write()` and `do_read()` methods. An alternative approach using the `uvm_reg_item` is described in the DVCon 2016 paper entitled "A New Class of Registers".

Example Stimulus

The following example sequence illustrates how the memory access methods could be used to implement a simple memory test.

```
//
// Test of memory 1
//
// Write to 10 random locations within the memory storing the data
// written
// then read back from the same locations checking against
// the original data
//
class mem_1_test_seq extends mem_ss_base_seq;

`uvm_object_utils(mem_1_test_seq)

rand uvm_reg_addr_t addr;

// Buffers for the addresses and the write data
uvm_reg_addr_t addr_array[10];
uvm_reg_data_t data_array[10];

function new(string name = "mem_1_test_seq");
    super.new(name);
endfunction

// The base sequence sets up the register model handle
task body;
    super.body();
    // Write loop
    for(int i = 0; i < 10; i++) begin
        // Constrain address to be within the memory range:
```

```

    if(!this.randomize() with {addr <= mem_ss_rm.mem_1.get_size();}) begin
        `uvm_error("body", "Randomization failed")
    end
    mem_ss_rm.mem_1.write(status, addr, data, .parent(this));
    addr_array[i] = addr;
    data_array[i] = data;
end
// Read loop
for(int i = 0; i < 10; i++) begin
    mem_ss_rm.mem_1.read(status, addr_array[i], data, .parent(this));
    if(data_array[i][31:0] != data[31:0]) begin
        `uvm_error("mem_1_test", $sformatf("Memory access error: expected
%0h, actual %0h", data_array[i][31:0], data[31:0]))
    end
end
endtask: body

endclass: mem_1_test_seq

```

Register Sequence Examples

To illustrate how the different register model access methods can be used from sequences to generate stimulus, this page contains a number of example sequences developed for stimulating the SPI master controller DUT.

Note that all of the example sequences that follow do not use all of the argument fields available in the methods. In particular, they do not use the map argument, since the access to the bus agent is controlled by the layering. If a register can be accessed by more than one bus interface, it will appear in several maps, possibly at different offsets. When the register access is made, the model selects which bus will be accessed. Writing the sequences this way makes it easier to reuse or retarget them in another integration scenario where they will access the hardware over a different bus infrastructure.

The examples shown are all derived from a common base sequence class template.

Register Sequence Base Class

In order to use the register model effectively with sequences, a base sequence needs to be written that takes care of getting the handle to the model and has data and status properties which are used by the various register access methods. This base class is then extended to implement sequences which use the register model.

The data field in the base class uses a register model type called `uvm_reg_data_t` which defaults to a 64 bit variable. The status field uses a register model enumerated type called `uvm_status_e`.

```

package spi_bus_sequence_lib_pkg;

import uvm_pkg::*;
`include "uvm_macros.svh"

import spi_env_pkg::*;
import spi_reg_pkg::*;

```

```

// Base class that used by all the other sequences in the
// package:
//
// Gets the handle to the register model - spi_rm
//
// Contains the data and status fields used by most register
// access methods
//
class spi_bus_base_seq extends uvm_sequence #(uvm_sequence_item);

`uvm_object_utils(spi_bus_base_seq)

// SPI Register model:
spi_reg_block spi_rm;
// SPI env configuration object (containing a register model handle)
spi_env_config m_cfg;

// Properties used by the various register access methods:
rand uvm_reg_data_t data; // For passing data
uvm_status_e status;      // Returning access status

function new(string name = "spi_bus_base_seq");
    super.new(name);
endfunction

// Common functionality:
// Getting a handle to the register model
task body;
    if(!uvm_config_db #(spi_env_config)::get(null, get_full_name(),
"spi_env_config", m_cfg)) begin
        `uvm_error("body", "Could not find spi_env_config")
    end
    spi_rm = m_cfg.spi_rm;
endtask: body

endclass: spi_bus_base_seq

```

Sequence using write

The following simple sequence illustrates the write method being used to set a register to a known value, in this case setting the SPI ss register to 0.

```

// Slave Unselect setup sequence
//
// Writes 0 to the slave select register
//
class slave_unselect_seq extends spi_bus_base_seq;

`uvm_object_utils(slave_unselect_seq)

```



```

function new(string name = "slave_unselect_seq");
    super.new(name);
endfunction

task body;
    super.body();
    spi_rm.ss_reg.write(status, 32'h0, .parent(this));
endtask: body

endclass: slave_unselect_seq

```

Sequence using randomize and set followed by update and get

The following sequence loads up the SPI control register with random data. In order to make the sequence reusable, the interrupt enable bit is a variable in the sequence.

The sequence randomizes the control register with a constraint that controls the size of the data word. Then the set method is used to setup the interrupt enable bit (ie) and to set the go_bsy bit to 0. Once the desired value has been setup, the update() method is called and this causes a bus level write to occur.

The final get() call in the sequence is used to update the sequence data variable from the control registers mirror value which has been updated by the predictor at the end of the transfer. This is done so that the virtual sequence controlling the generation process can store the current configuration of the control register.

```

//
// Ctrl set sequence - loads one control params
//                      but does not set the go bit
//
class ctrl_set_seq extends spi_bus_base_seq;

    `uvm_object_utils(ctrl_set_seq)

    function new(string name = "ctrl_set_seq");
        super.new(name);
    endfunction

    // Controls whether interrupts are enabled or not
    bit int_enable = 0;

    task body;
        super.body();
        // Constrain to interesting data length values
        if(!spi_rm.ctrl_reg.randomize() with {char_len.value inside {0, 1,
[31:33], [63:65], [95:97], 126, 127}}) begin
            `uvm_error("body", "Control register randomization failed")
        end
        // Set up interrupt enable
        spi_rm.ctrl_reg.ie.set(int_enable);
        // Don't set the go_bsy bit
        spi_rm.ctrl_reg.go_bsy.set(0);
    endtask
endclass

```

```

// Write the new value to the control register
spi_rm.ctrl_reg.update(status, .path(UVM_FRONTDOOR), .parent(this));
// Get a copy of the register value for the SPI agent
data = spi_rm.ctrl_reg.get();
endtask: body

endclass: ctrl_set_seq

```

Sequence using read and write, peek and poke

The SPI register test sequence illustrates the use of the read() and write() methods, followed by the peek() and poke() methods. It also illustrates a few other features of the register model which can prove useful for similar types of tests. The register test sequence starts by doing a read from all the registers to check their reset values, then it writes a random value to each register in a random order, followed by a read that checks that the value read back is the same as the value written. In order to access all the registers in the SPI block, the block level get_registers() method is called. This returns an array of handles for all the registers in the block, and subsequent accesses to the registers are made using this array.

In the reset test, the reset value for each register is copied into the ref_data variable using the get_reset() method. Then the register is read and the returned data value compared against the ref_data variable.

In the write/read test, the base sequences data variable is randomized and then written to the selected register. Then the register handle array is shuffled and the get() method is used to copy the registers mirrored value into ref_data to be compared against the real hardware value via the read() method.

In the peek and poke test, the same test strategy is used, but this time using the backdoor access.

Note that in all this activity, the register address is not used. Only in the write loop is there a check on the name of the register to ensure that if it is the control register that the bit that initiates the SPI transfer is not set.

```

//
// This is a register check sequence
//
// It checks the reset values
//
// Then it writes random data to each of the registers
// and reads back to check that it matches
//
class check_regs_seq extends spi_bus_base_seq;

`uvm_object_utils(check_regs_seq)

function new(string name = "check_regs_seq");
    super.new(name);
endfunction

uvm_reg spi_regs[$];
uvm_reg_data_t ref_data;

task body;

    super.body;

```

```

spi_rm.get_registers(spi_regs);

// Read back reset values in random order
spi_regs.shuffle();
foreach(spi_regs[i]) begin
    ref_data = spi_regs[i].get_reset(); spi_regs[i].read(status, data,
    .parent(this)); if(ref_data != data) begin
        `uvm_error("REG_TEST_SEQ:", $sformatf("Reset read error for %s: Expected: %0h Actual:
%0h", spi_regs[i].get_name(), ref_data, data))
    end
end

// Write random data and check read back (10 times)
repeat(10) begin

    spi_regs.shuffle();
    foreach(spi_regs[i]) begin assert(this.randomize());
        if(spi_regs[i].get_name() == "ctrl") begin
            data[8] = 0;
        end
        spi_regs[i].write(status, data, .parent(this));
    end
    spi_regs.shuffle();
    foreach(spi_regs[i]) begin
        ref_data = spi_regs[i].get(); spi_regs[i].read(status, data,
        .parent(this)); if(ref_data != data) begin
            `uvm_error("REG_TEST_SEQ:", $sformatf("get/read: Read error for
%s: Expected: %0h Actual: %0h", spi_regs[i].get_name(), ref_data, data))
        end
    end
end

// Repeat with back door accesses
repeat(10) begin spi_regs.shuffle();
    foreach(spi_regs[i]) begin
        assert(this.randomize()); if(spi_regs[i].get_name() == "ctrl")
        begin
            data[8] = 0;
        end
        spi_regs[i].poke(status, data, .parent(this));
    end
end

```

```

spi_regs.shuffle();
foreach(spi_regs[i]) begin
    ref_data = spi_regs[i].get();
    spi_regs[i].peek(status, data, .parent(this));
    if(ref_data[31:0] != data[31:0]) begin
        `uvm_error("REG_TEST_SEQ:", $sformatf("poke/peek: Read error
for %s: Expected: %0h Actual: %0h", spi_regs[i].get_name(), ref_data,
data))
    end
    spi_regs[i].read(status, data, .parent(this));
end

end

endtask: body

endclass: check_regs_seq

```

Sequence using mirror

The mirror() method causes a read access to take place which updates the mirrored value in the register, it does not return the read data. This can be useful to re-sync the register model with the hardware state. The mirror() method can be used, together with a predict() call, to check that the data read back from a register matches the expected data.

The following sequence uses the mirror() method to check that the data read back from the target hardware registers matches the expected values which have been set in the register mirrored values. In this case, the mirrored values have been set via the testbench scoreboard according to the data monitored on the input bus. The mirror() method call has its check field set to UVM_CHECK which ensures that the data actually read back matches that predicted by the scoreboard. If the check field is at its default value, the mirror() method would simply initiate a bus read cycle which would result in the external predictor updating the mirror value.

```

//
// Data unload sequence - reads back the data rx registers
// all of them in a random order
//
class data_unload_seq extends spi_bus_base_seq;

    `uvm_object_utils(data_unload_seq)

    uvm_reg data_regs[];

    function new(string name = "data_unload_seq");
        super.new(name);
    endfunction

    task body;
        super.body();
        // Set up the data register handle array
        data_regs = '{spi_rm.rxtx0_reg, spi_rm.rxtx1_reg, spi_rm.rxtx2_reg,

```

```
spi_rm.rxtx3_reg};  
    // Randomize access order  
    data_regs.shuffle();  
    // Use mirror in order to check that the value read back is as  
    expected  
    foreach(data_regs[i]) begin  
        data_regs[i].mirror(status, UVM_CHECK, .parent(this));  
    end  
endtask: body  
  
endclass: data_unload_seq
```

Note that this example of a sequence interacting with a scoreboard is not a recommended approach, it is provided as a means of illustrating the use of the mirror() method.

Built-in Register Sequences

The UVM package contains a library of automatic test sequences which are based on the register model. These sequences can be used to do basic tests on registers and memory regions within a DUT. The automatic tests are aimed at testing basic functionality such as checking register reset values are correct or that the read-write data paths are working correctly. One important application of these sequences is for quick sanity checks when bringing up a sub-system or SoC design where a new interconnect or address mapping scheme needs to be verified.

Registers and memories can be opted out of these auto tests by setting an individual "DO_NOT_TEST" attribute which is checked by the automatic sequence as runs. An example of where such an attribute would be used is a clock control register where writing random bits to it will actually stop the clock and cause all further DUT operations to fail.

The register sequences which are available within the UVM package are summarized in the following tables.

Note: that any of the automatic tests can be disabled for a given register or memory by the **NO_REG_TEST** attribute, or for memories by the **NO_MEM_TEST** attribute. The disable attributes given in the table are specific to the sequences concerned.

Disclaimer: The built in register sequences do not return a pass or fail status, if you use them, then you need to take responsibility for checking the result. In early bring up testing, this might be done by manually checking the result, in later testing, scoreboarding will be required.

Register Built In Sequences

Sequence	Disable Attribute	Block Level	Register Level	Description
uvm_reg_hw_reset_seq	NO_REG_HW_RESET_TEST	Yes	Yes	Checks that the Hardware register reset value matches the value specified in the register model
uvm_reg_single_bit_bash_seq	NO_REG_BIT_BASH_TEST	No	Yes	Writes, then check-reads 1's and 0's to all bits of the selected register that have read-write access
uvm_reg_bit_bash_seq	NO_REG_BIT_BASH_TEST	Yes	No	Executes the uvm_reg_single_bit_bash_seq for each register in the selected block and any sub-blocks it may contain
uvm_reg_single_access_seq	NO_REG_ACCESS_TEST	No	Yes	Writes to the selected register via the frontdoor, checks the value is correctly written via the backdoor, then writes a value via the backdoor and checks that it can be read back correctly via the frontdoor. Repeated for each address map that the register is present in. Requires that the backdoor hdl_path has been specified
uvm_reg_access_seq	NO_REG_ACCESS_TEST	Yes	No	Executes the uvm_reg_single_access_seq for each register accessible via the selected block
uvm_reg_shared_access_seq	NO_SHARED_ACCESS_TEST	No	Yes	For each map containing the register, writes to the selected register in one map, then check-reads it back from all maps from which it is accessible. Requires that the selected register has been added to multiple address maps.

Some of the register test sequences are designed to run on an individual register, whereas some are block level sequences which go through each accessible register and execute the single register sequence on it.

Memory Built In Sequences

Sequence	Disable Attributes	Block Level	Memory Level	Description
uvm_mem_single_walk_seq	NO_MEM_WALK_TEST	No	Yes	Writes a walking pattern into each location in the range of the specified memory, then checks that is read back with the expected value
uvm_mem_walk_seq	NO_MEM_WALK_TEST	Yes	No	Executes uvm_mem_single_walk_seq on all memories accessible from the specified block
uvm_mem_single_access_seq	NO_MEM_ACCESS_TEST	No	Yes	For each location in the range of the specified memory: Writes via the frontdoor, checks the value written via the backdoor, then writes via the backdoor and reads back via the front door. Repeats test for each address map containing the memory. Requires that the backdoor hdl_path has been specified.
uvm_mem_access_seq	NO_MEM_ACCESS_TEST	Yes	No	Executes uvm_mem_single_access_seq for each memory accessible from the specified block

uvm_mem_shared_access_seq	NO_SHARED_ACCESS_TEST	No	Yes	For each map containing the memory, writes to all memory locations and reads back from all using each of the address maps. Requires that the memory has been added to multiple address maps.
---------------------------	-----------------------	----	-----	---

Like the register test sequences, the tests either run on an individual memory basis, or on all memories contained within a block. Note that the time taken to execute a memory test sequence could be lengthy with a large memory range.

Aggregated Register And Memory Built In Sequences

These sequences run at the block level only

Sequence	Disable Attributes	Description
uvm_reg_mem_shared_access_seq	NO_SHARED_ACCESS_TEST	Executes the uvm_reg_shared_access_reg_seq on all registers accessible from the specified block. Executes the uvm_mem_shared_access_seq on all memories accessible from the specified block
uvm_reg_mem_built_in_seq	All of the above	Executes all of the block level auto-test sequences
uvm_reg_mem_hdl_paths_seq	None	Used to test that any specified HDL paths defined within a block are accessible by the backdoor access. The check is only performed on registers or memories which have HDL paths declared.

Setting An Attribute

In order to set an auto-test disable attribute on a register, you will need to use the UVM resource_db to set a bit with the attribute string, giving the path to the register or the memory as the scope variable. Since the UVM resource database is used, the attributes can be set from anywhere in the testbench at any time. However, the recommended approach is to set the attributes as part of the register model, this will most likely be done by specifying the attribute via the register model generators specification input file.

The following code excerpt shows how attributes would be implemented in a register model.

```
// From the build() method of the memory sub-system (mem_ss) block:
function void build();
    //
    // .....
    //
    // Example use of "dont_test" attributes:
    // Stops mem_1_offset reset test
    uvm_resource_db #(bit)::set({"REG::",
this.mem_1_offset.get_full_name()}, "NO_REG_HW_RESET_TEST", 1);
    // Stops mem_1_offset bit-bash test
    uvm_resource_db #(bit)::set({"REG::",
this.mem_1_offset.get_full_name()}, "NO_REG_BIT_BASH_TEST", 1);
    // Stops mem_1 being tested with the walking auto test
    uvm_resource_db #(bit)::set({"REG::", this.mem_1.get_full_name()},
"NO_MEM_WALK_TEST", 1);
    lock_model();
endfunction: build
```

This shows how the same could be achieved from a sequence:

```
//
// From within a sequence where the mem_ss_rm handle is set in the
base_class:
//
task body;
    super.body();
    // Disable mem_2 walking auto test
    uvm_resource_db #(bit)::set({"REG:" ,
mem_ss_rm.mem_1.get_full_name()}, "NO_MEM_WALK_TEST", 1);
    //
    // ...
    //
endtask: body
```

Note that once an attribute has been set in the UVM resource database, it cannot be 'unset'. This means that successive uses of different levels of disabling within sequences may produce unwanted accumulative effects.

Built In Sequence Example

The example sequence is from a testbench for a memory sub-system which contains some registers which set up the address decoding for the sub-system memory arrays and then control accesses to them. It uses 3 of the built-in test sequences, two to test the register and the other to test all of the memory arrays in the sub-system.

In order for any of the UVM automatic tests to run, they need to have their register model handle assigned to the register model being used. They all use the name 'model' for the register model handle. This is illustrated in the example.

```
//
// Auto test of the memory sub-system using the built-in
// automatic sequences:
//
class auto_tests extends mem_ss_base_seq;

`uvm_object_utils(auto_tests)

function new(string name = "auto_tests");
    super.new(name);
endfunction

task body;
    // Register reset test sequence
    uvm_reg_hw_reset_seq rst_seq =
uvm_reg_hw_reset_seq::type_id::create("rst_seq");
    // Register bit bash test sequence
    uvm_reg_bit_bash_seq reg_bash =
uvm_reg_bit_bash_seq::type_id::create("reg_bash");
    // Initialise the memory mapping registers in the sub-system
    mem_setup_seq setup = mem_setup_seq::type_id::create("setup");
```



```
// Memory walk test sequence
uvm_mem_walk_seq walk = uvm_mem_walk_seq::type_id::create("walk");

super.body(); // Gets the register model handle
// Set the register model handle in the built-in sequences
rst_seq.model = mem_ss_rm;
walk.model = mem_ss_rm;
reg_bash.model = mem_ss_rm;

// Start the test sequences:
//
// Register reset:
rst_seq.start(m_sequencer);
// Register bit-bash
reg_bash.start(m_sequencer);
// Set up the memory sub-system
setup.start(m_sequencer);
// Memory walk test
walk.start(m_sequencer);

endtask: body

endclass: auto_tests
```

Example Download

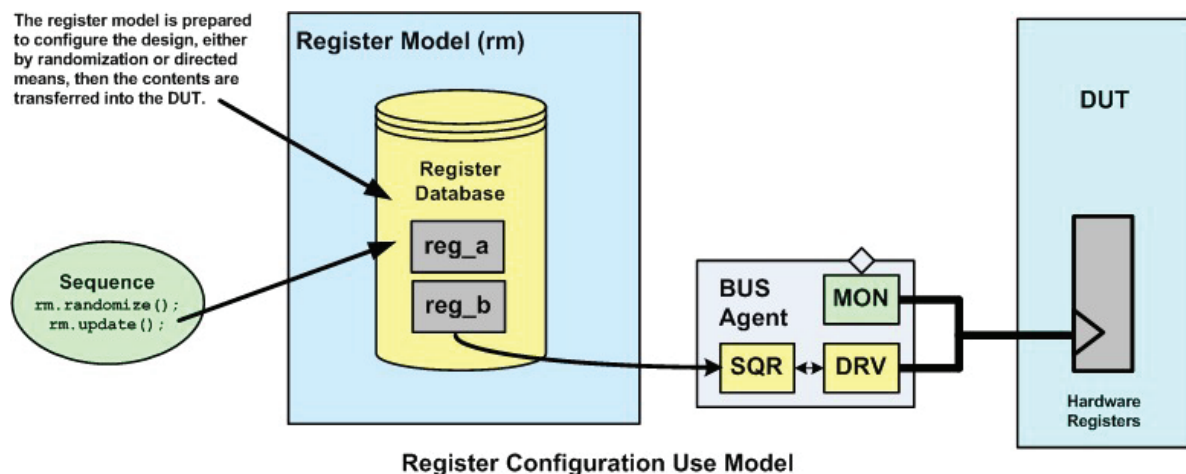
The code for this example can be downloaded at the verificationacademy.com website

Configuring Registers

During verification a programmable hardware device needs to be configured to operate in different modes. The register model can be used to automate or to semi-automate this process.

The register model contains a shadow of the register state space for the DUT which is kept up to date as the design is configured using bus read and write cycles. One way to configure the design in a testbench is to apply reset to the design and then go through a programming sequence which initializes the design for a particular mode of operation.

In real life, a programmable device might be used for a while in one mode of operation, then reconfigured to be used in another mode and the non re-programmed registers will effectively have random values. Always initializing from reset has the shortcoming that the design always starts from a known state and it is possible that a combination of register values that causes a design failure would be missed. However, if the register map is randomized at the beginning of the simulation and the randomized contents of the register map are written to the DUT before configuring it into the desired mode, it is possible to emulate the conditions that would exist in a 'mid-flight' re-configuration.



Register Configuration Use Model

The register model can be used to configure the design by creating a configuration state 'off-line' using whatever mixture of constrained randomization or directed programming is convenient. If the register model desired values are updated, then the transfer of the configuration to the DUT can be done using the update() method, this will transfer any new values that need to be written to the hardware in the order they are declared in the register model.

The transfer of the register model to the DUT can be done either in an ordered way, or it can be done in a random order. Some designs require that at least some of the register programming is done in a particular order.

In order to transfer the data in a random order, the registers in the model should be collected into an array and then the array should be shuffled:

```
//
// Totally random configuration
//
task body;
    uvm_reg spi_regs[];

    super.body();
    spi_rm.get_registers(spi_regs);
    if(!spi_rm.randomize()) begin
        `uvm_error("body", "spi_rm randomization failed")
    end
end
```

```

spi_regs.shuffle(); // Randomly re-order the array
foreach(spi_regs[i]) begin
    spi_regs[i].update(); // Only change the reg if required
end
endtask: body

```

Here is an example of a sequence that configures the SPI using the register model. Note that it uses constraints to configure the device within a particular range of operation, and that the write to the control register is a setup write which will be followed by an enabling write in another sequence.

```

//
// Sequence to configure the SPI randomly
//
class SPI_config_seq extends spi_bus_base_seq;

`uvm_object_utils(SPI_config_seq)

function new(string name = "SPI_config_seq");
    super.new(name);
endfunction

bit interrupt_enable;

task body;
    super.body;

    // Randomize the register model to get a new config
    // Constraining the generated value within ranges
    if(!spi_rm.randomize() with {spi_rm.ctrl_reg.go_bsy.value == 0;
                                spi_rm.ctrl_reg.ie.value ==
interrupt_enable;
                                spi_rm.ss_reg.cs.value != 0;
                                spi_rm.ctrl_reg.char_len.value inside {0, 1,
[31:33], [63:65], [95:97], 126, 127};
                                spi_rm.divider_reg.ratio.value inside {0, 1,
2, 4, 8, 16, 32, 64, 128};
                                }) begin
        `uvm_error("body", "spi_rm randomization failure")
    end
    // This will write the generated values to the HW registers
    spi_rm.update(status, .path(UVM_FRONTDOOR), .parent(this));
    data = spi_rm.ctrl_reg.get();
endtask: body

endclass: SPI_config_seq

```

A DUT could be reconfigured multiple times during a test case in order to find unintended interactions between register values.

Register-Level Scoreboards

The UVM register model shadows the current configuration of a programmable DUT and this makes it a valuable resource for scoreboards that need to be aware of the current DUT state. The scoreboard references the register model to either adapt its configuration to match the current state of a programmable DUT or to adapt its algorithms in order to make the right comparisons. For instance, checking that a communications device with a programmable packet configuration has transmitted or received the right data requires the scoreboard to know the current data format.

The UVM register model will contain information that is useful to the scoreboard:

- DUT Configuration, based on the register state
- Data values, based on the current state of buffer registers

The UVM register model can also be used by the scoreboard to check DUT behavior:

- A register model value can be set up to check for the correct expected value
- A backdoor access can check the actual state of the DUT hardware - This can be useful if there is volatile data that causes side effects if it is read via the front door.

Register Model Access

In order to reference the contents of the register model, the scoreboard will require its handle. This can be assigned either from a configuration object or by using a resource from the UVM resource database. Once this handle is assigned, then the contents of the register database can be accessed. Field or register values can either be accessed by reference using a path to the mirrored field value (e.g. `spi_rm.ctrl.ie.value`) or by calling the `get_mirrored_value()` access method (e.g. `spi_rm.ctrl.ie.get_mirrored_value()`). This configuration information can then be used to inform decisions in the scoreboard logic.

Checking DUT Behaviour

DUT register contents can be checked in one of several ways.

The simplest way is to compare an observed output against the original data. For instance, a scoreboard for a parallel to serial converter function could compare the data in an observed serial output packet against the data in the transmit buffer registers.

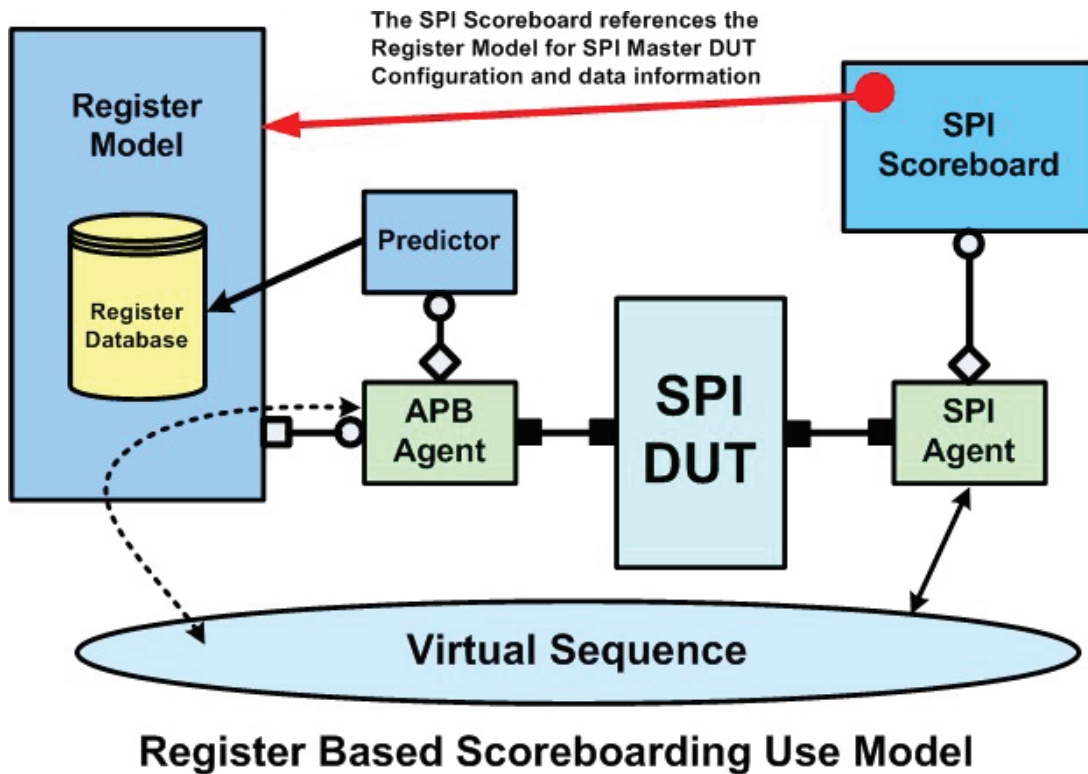
Another way is to use the register model to check the contents of the DUT against their expected contents. For instance, in the case of a serial to parallel converter, the scoreboard could determine the expected contents of the receive buffer, and use a back door `peek()` access of the hardware receive buffer to determine that the correct data has been received.

An alternative to using `peek()` would be to use the `predict()` register access method to set the mirrored values and then access the DUT registers using the `mirror()` access function with its `check` argument set. The `mirror()` call will detect and report any errors due to a mismatch between the expected and actual data. The `mirror()` accesses could be made from the scoreboard, or they could be executed from a supporting sequence.

Note: For information on `peek()`, `mirror()` and other register model access methods mentioned on this page, see the material in the [Registers/StimulusAbstraction](#) article.

Example register model based scoreboard

The SPI scoreboard uses the SPI register model in several of the ways described above. In order to scoreboard the SPI master DUT, it is necessary to check its configuration. In this case, the SPI master DUT is capable of transferring a variable number of bits in and out of the data buffer in either LSB or MSB first order. It can also transmit on different clock edges and sample receive data on different clock edges. This configuration information is looked up in the SPI register model so that the scoreboard logic is configured correctly.



Functional Overview

The scoreboard reacts to analysis transactions from the SPI agent. These correspond to a completed transfer, and the transmitted (Master Out Slave In -MOSI) data is compared against the data in the transmit buffer in the register model before the received (Master In Slave Out - MISO) data is compared against the data peeked from the DUT receive buffer. The RXTX register mirrored value is also updated using the predict() method, this allows any subsequent front door access using a checking mirror() method call to check the read data path from these registers.

The SPI scoreboard also checks that the SPI chip select values used during the transfer match the value programmed in the slave select (SS) register.

The following excerpts from the scoreboard code illustrate how the scoreboard references the register model.

Register Model Handle

The SPI scoreboard needs to refer to the spi_rm register model and contains a handle to do so. This handle is assigned in its containing env from the register model handle in the env's configuration object:

```
function void spi_env::connect();
//
  if(m_cfg.has_spi_scoreboard) begin
    m_spi_agent.ap.connect(m_scoreboard.spi.analysis_export); // SPI
  end
  Monitor
```

```

    m_scoreboard.spi_rm = m_cfg.spi_rm; // SPI Register Model
end
//
endfunction: connect

```

Referring to the Register Model To Check SPI TX (MOSI) Data

The SPI master DUT can transmit up to 128 bits of data, and the transmit order can be either LSB or MSB first. The data can be changed on either the rising or the falling edge of the SPI clock. The SPI agent monitors the SPI data transfer and sends an analysis transaction every time a SPI character transfer completes. The scoreboard uses the content of the SPI analysis transaction, together with the configuration details from the register model to compare the observed transmit data against the data written into the TX data registers.

```

forever begin
    error = 0;
    spi.get(item);
    no_transfers++;
    // Get the character length from the register model
    //
    // char_len is the character length field in the ctrl register
    //
    bit_cnt = spi_rm.ctrl_reg.char_len.get_mirrored_value();
    // Corner case for bit count equal to zero:
    if(bit_cnt == 8'b0) begin
        bit_cnt = 128;
    end
    // Get the transmit data from the register model
    //
    // The 128 bits of tx data is in 4 registers rctx0-rctx3
    //
    tx_data[31:0] = spi_rm.rctx0_reg.get_mirrored_value();
    tx_data[63:32] = spi_rm.rctx1_reg.get_mirrored_value();
    tx_data[95:64] = spi_rm.rctx2_reg.get_mirrored_value();
    tx_data[127:96] = spi_rm.rctx3_reg.get_mirrored_value();

    // Find out if the tx (mosi) data is sampled on the neg or pos edge
    //
    // The ctrl register tx_neg field contains this info
    //
    // The SPI analysis transaction (item) contains samples for both
edges
    //
    if(spi_rm.ctrl_reg.tx_neg.get_mirrored_value() == 1) begin
        mosi_data = item.nedge_mosi; // To be compared against write data
    end
    else begin
        mosi_data = item.pedge_mosi;
    end
    //

```

```

    // Compare the observed MOSI bits against the tx data written to
the SPI DUT
    //
    // Find out whether the MOSI data is transmitted LSB or MSB first -
    this
    // affects the comparison
    if(spi_rm.ctrl_reg.lsb.get_mirrored_value() == 1) begin // LSB
first
        for(int i = 0; i < bit_cnt; i++) begin if(tx_data[i] !=
            mosi_data[i]) begin
                error = 1;
            end
        end
        if(error == 1) begin
            `uvm_error("SPI_SB_MOSI_LSB:", $sformatf("Expected mosi value
%0h actual %0h", tx_data, mosi_data))
        end
    end
    else begin // MSB first
        for(int i = 0; i < bit_cnt; i++) begin
            if(tx_data[i] != mosi_data[(bit_cnt-1) - i]) begin
                error = 1;
            end
        end
        if(error == 1) begin // Need to reverse the mosi_data bits for
the error message
            rev_miso = 0;
            for(int i = 0; i < bit_cnt; i++) begin
                rev_miso[(bit_cnt-1) - i] = mosi_data[i];
            end
            `uvm_error("SPI_SB_MOSI_MSB:", $sformatf("Expected mosi value
%0h actual %0h", tx_data, rev_miso))
        end
    end
    if(error == 1) begin
        no_tx_errors++;
    end
    //
    // TX Data checked

```

Referring to the Register Model To Check SPI RX (MISO) Data

The SPI slave (RX) data will follow the same format as the transmit data, but it could be sampled on the opposite edge of the SPI clock, this is determined by a control bit field in the SPI control register. The scoreboard looks up this information, manipulates the MISO data in the SPI analysis transaction as necessary to get it in the right order and then compares it against the value read back by a series of back door peek()s from the SPI data registers.

Although not strictly necessary, the scoreboard calls the predict() method for each of the data registers using the data observed via the peek() method. This sets things up so that if a sequence makes a mirror() call to these registers with a check enabled, the return data path will be checked.

```
// RX Data check
// Reset the error bit
error = 0;
// Check the miso data (RX)
//
// Look up in the register model which edge the RX data should be
sampled on
//
if(spi_rm.ctrl_reg.rx_neg.get_mirrored_value() == 1) begin
    miso_data = item.pedge_miso;
end
else begin
    miso_data = item.nedge_miso;
end
// Reverse the order of the observed RX data bits if MSB first
//
// Determine this by looking up the ctrl.lsb in the register model
//
if(spi_rm.ctrl_reg.lsb.get_mirrored_value() == 0) begin // MSB
    // reverse the bits lsb -> msb, and so on
    rev_miso = 0;
    for(int i = 0; i < bit_cnt; i++) begin
        rev_miso[(bit_cnt-1) - i] = miso_data[i];
    end
    miso_data = rev_miso;
end

// The following sets up the rx data so that it is
// bit masked according to the no of bits
rx_data = spi_rm.rxtx0_reg.get_mirrored_value();
// Peek the RX data in the hardware and compare against the
observed RX data
spi_rm.rxtx0_reg.peek(status, spi_peek_data);
for(int i = 0; ((i < 32) && (i < bit_cnt)); i++) begin
    rx_data[i] = miso_data[i];
    if(spi_peek_data[i] != miso_data[i]) begin
        error = 1;
        `uvm_error("SPI_SB_RXD:", $sformatf("Bit%0d Expected RX data
```



```

value %0h actual %0h", i, spi_peek_data[31:0], miso_data))
    end
end
    // Get the register model to check that the data it next reads back
from this
    // register is as predicted - this is done by a sequence calling
the mirror()
    // method with a check enabled
    //
    // This is somewhat redundant given the earlier peek check, but it
does check the
    // read back path
    assert(spi_rm.rtx0_reg.predict(rx_data));

    // Repeat for any remaining bits with the rtx1, rtx2, rtx3
registers

```

Checking the SPI Chip Selects

The final check in the scoreboard is that the observed value of the SPI slave chip select lines corresponds to the value programmed in the SPI SS registers cs (chip select) field.

```

// Check the chip select lines
//
// Compare the programmed value of the SS register (i.e. its cs field)
against
// the cs bit pattern observed on the SPI bus
//
if(spi_rm.ss_reg.cs.get_mirrored_value() != {56'h0, ~item.cs}) begin
    `uvm_error("SPI_SB_CS:", $sformatf("Expected cs value %b actual %b",
spi_rm.ss_reg.cs.get(), ~item.cs))
    no_cs_errors++;
end

```

The scoreboard code can be found in the *spi_scoreboard.svh* file in the env sub-directory of the SPI block level testbench example which can be downloaded via the link below:

Register-Level Functional Coverage

Register Based Functional Coverage Overview

The UVM supports the collection of functional coverage based on register state in three ways:

- Automatic collection of register coverage based on covergroups inside the register model on each access
- Controlled collection of register coverage based on covergroups inside the register model by calling a method from outside the register model
- By reference, from an external covergroup that samples register value via a register model handle

Most register model generators allow users to specify the automatic generation of cover groups based on bit field or register content. These are fine if you have a narrow bit field and you are interested in all the states that the field could take, but they quickly lose value and simply add simulation overhead for minimal return. In order to gather register based functional coverage that is meaningful, you will need to specify coverage in terms of a cross of the contents of several registers and possibly non register signals and/or variables. Your register model generation may help support this level of complexity but, if not, it is quite straight-forward to implement an external functional coverage collection component that references the register model.

The recommended approach is to use an external covergroup that samples register values via the register model handle.

Controlling Register Model Functional Coverage Collection

A register model may contain many covergroups and this has the potential to have a serious impact on simulation performance. Therefore there are various inter-locks built into the register model which allow you to specify which type of coverage model you want to use and to enable or disable coverage collection during the course of a test case. A bit mapped enumerated type is used to enable the different coverage models and the available options are:

enum value	Coverage Enabled
UVM_NO_COVERAGE	None, all coverage disabled
UVM_CVR_REG_BITS	Collect coverage for bits read from or written to in registers
UVM_CVR_ADDR_MAP	Collect coverage for addresses read from or written to in address maps
UVM_CVR_FIELD_VALS	Collect coverage for the values stored in register fields
UVM_CVR_ALL	Collect all coverage

The bit mapped enumeration allows several coverage models to be enabled in one assignment by logically ORing several different values - e.g. `set_coverage(UVM_CVR_ADDR_MAP + UVM_CVR_FIELD_VALS)`

A register model can contain coverage groups which have been assigned to each of the active categories and the overall coverage for the register model is set by a static method in the `uvm_reg` class called `include_coverage()`. This method should be called before the register model is built, since it creates an entry in the resource database which the register model looks up during the execution of its `build()` method to determine which covergroups to build.

```
//
//From the SPI test base
//
// Build the env, create the env configuration
// including any sub configurations and assigning virtual interfaces
function void spi_test_base::build_phase(uvm_phase build);
    // env configuration
```

```

m_env_cfg = spi_env_config::type_id::create("m_env_cfg");
// Register model
// Enable all types of coverage available in the register model
uvm_reg::include_coverage("*", UVM_CVR_ALL);
// Create the register model:
spi_rm = spi_reg_block::type_id::create("spi_rm");
// Build and configure the register model
spi_rm.build();

```

As the register model is built, coverage sampling is enabled for the different coverage categories that have been enabled. The coverage sampling for a category of covergroups within a register model hierarchical object can then be controlled using the `set_coverage()` method in conjunction with the `has_coverage()` method (which returns a value corresponding to the coverage categories built in the scope) and the `get_coverage()` method (which returns a value corresponding to the coverage model types that are currently being actively sampled).

For more detail on how to implement a register model so that it complies with the build and control structure for covergroups see `ModelCoverage`.

Register Model Coverage Sampling

The covergroups within the register model will in most cases be defined by the model specification and generation process and the end user may not know how they are implemented. The covergroups within the register model can be sampled in one of two ways.

Some of the covergroups in the register model are sampled as a side-effect of a register access and are therefore automatically sampled. For each register access, the automatic coverage sampling occurs in the register and in the block that contains it. This type of coverage is important for getting coverage data on register access statistics and information which can be related to access of a specific register.

Other covergroups in the register model are only sampled when the testbench calls the `sample_values()` method from a component or a sequence elsewhere in the testbench. This allows more specialized coverage to be collected. Potential applications include:

- Sampling register state (DUT configuration) when a specific event such as an interrupt occurs
- Sampling register state only when a particular register is written to

Referencing The Register Model Data In External Functional Coverage Monitors (Recommended)

An alternative way of implementing register based functional coverage is to build a functional coverage monitor component separate from the register model, but to sample values within the register model. The advantages of this approach are:

- The covergroup(s) within the external monitor can be developed separately from the register model implementation
- The sampling of the covergroup(s) can be controlled more easily
- It is possible to mix, or cross, sampled contents of the register model with the sampled values of other testbench variables

The following example shows a functional coverage monitor from the SPI testbench which references the SPI register model.

```

class spi_reg_functional_coverage extends uvm_subscriber
#(apb_seq_item);

```

```

`uvm_component_utils(spi_reg_functional_coverage)

logic [4:0] address;
bit wnr; spi_reg_block spi_rm;

// Checks that the SPI master registers have
// all been accessed for both reads and writes
covergroup reg_rw_cov;
  option.per_instance = 1; ADDR:
  coverpoint address {
    bins DATA0 = {0}; bins
    DATA1 = {4}; bins DATA2 =
    {8}; bins DATA3 = {5'hC};
    bins CTRL = {5'h10};
    bins DIVIDER = {5'h14};
    bins SS = {5'h18};
  }
  CMD: coverpoint wnr {
    bins RD = {0};
    bins WR = {1};
  }
  RW_CROSS: cross CMD, ADDR;
endgroup: reg_rw_cov

//
// Checks that we have tested all possible modes of operation
// for the SPI master
//
// Note that the field value is 64 bits wide, so only the relevant
// bit(s) are used
covergroup combination_cov;

  option.per_instance = 1;

  ACS: coverpoint spi_rm.ctrl_reg.acs.value[0]; IE: coverpoint
  spi_rm.ctrl_reg.ie.value[0]; LSB: coverpoint
  spi_rm.ctrl_reg.lsb.value[0];
  TX_NEG: coverpoint spi_rm.ctrl_reg.tx_neg.value[0]; RX_NEG:
  coverpoint spi_rm.ctrl_reg.rx_neg.value[0];
  // Suspect character lengths - there may be more
  CHAR_LEN: coverpoint spi_rm.ctrl_reg.char_len.value[6:0] {
    bins LENGTH[] = {0, 1, [31:33], [63:65], [95:97], 126, 127};
  }

```

```

CLK_DIV: coverpoint spi_rm.divider_reg.ratio.value[7:0] {
    bins RATIO[] = {16'h0, 16'h1, 16'h2, 16'h4, 16'h8, 16'h10, 16'h20,
16'h40, 16'h80};
}
COMB_CROSS: cross ACS, IE, LSB, TX_NEG, RX_NEG, CHAR_LEN, CLK_DIV;
endgroup: combination_cov

extern function new(string name = "spi_reg_functional_coverage",
uvm_component parent = null);
extern function void write(T t);

endclass: spi_reg_functional_coverage

function spi_reg_functional_coverage::new(string name =
"spi_reg_functional_coverage", uvm_component parent = null);
    super.new(name, parent);
    reg_rw_cov = new();
    combination_cov = new();
endfunction

function void spi_reg_functional_coverage::write(T t);
    // Register coverage first
    address = t.addr[4:0];
    wnr = t.we;
    reg_rw_cov.sample();
    // Sample the combination covergroup when go_bsy is true
    if(address == 5'h10)
        begin
            if(wnr) begin
                if(t.data[8] == 1) begin
                    combination_cov.sample(); // TX started
                end
            end
        end
    end
endfunction: write

```

Coding Guideline: Wrap covergroups within uvm_objects

A covergroup should be implemented within a wrapper class derived from a uvm_object.

Justification:

Wrapping a covergroup in this way has the following advantages:

- The uvm_object can be constructed at any time - and so the covergroup can be brought into existence at any time, this aids conditional deferred construction.
- The covergroup wrapper class can be overridden from the factory, which allows an alternative coverage model to be substituted if required.
- This advantage may become more relevant when different active phases are used in future.

Example:

```
class covergroup_wrapper extends uvm_object;

  `uvm_object_utils(covergroup_wrapper)

  covergroup cg (string name) with function sample(my_reg reg, bit
is_read);
    option.name = name;
    CHAR_LEN: coverpoint reg.char_len {
      bins len_5 = {2'b00};
      bins len_6 = {2'b01};
      bins len_7 = {2'b10};
      bins len_8 = {2'b11};
    }
    PARITY: coverpoint reg.parity {
      bins parity_on = {1'b1};
      bins parity_off = {1'b0};
    }
    ALL_OPTIONS: cross CHAR_LEN, PARITY;
  endgroup: cg

  function new(string name = "covergroup_wrapper");
    super.new(name);
    cg = new();
  endfunction

  function void sample(my_reg reg_in, bit is_read_in);
    cg.sample(my_reg reg_in, bit is_read_in);
  endfunction: sample

endclass: covergroup_wrapper
```